Extensiones de Machine Learning
Aprendizaje por Refuerzo

Marta Lucas Marín
m.lucasmarin@um.es

Francisco José Lucas Robles
fjlr1@um.es

Jorge Ruiz Ortiz
j.ruizortiz@um.es

15 de marzo de 2026

¡Nunca seré derrotado por una máquina!
Jamás se inventará un programa que
supere la inteligencia humana.

G. Kasparov, 1989[1]

[1]De una entrevista publicada en la edición nº 55 de la revista *Jeux & Stratégie*.

Resumen

Este trabajo estudia el Aprendizaje por Refuerzo (RL) de forma progresiva, partiendo del paradigma más simple de toma de decisiones secuencial hasta los métodos de aproximación profunda con redes neuronales, a través de tres bloques de contenido interconectados.

Capítulo 1 — Problema del Bandido de k-brazos. Se introduce el dilema fundamental del RL: el balance entre *exploración* y *explotación*. Un agente interactúa con un entorno estocástico de k palancas, cada una con una distribución de recompensa desconocida, buscando maximizar la ganancia acumulada en un horizonte finito de $T = 300$ pasos. Se implementan y comparan empíricamente tres familias algorítmicas bajo $N = 500$ ejecuciones y distribuciones de recompensa Normal, Binomial y Bernoulli: (i) **ϵ-Greedy** y su variante con decaimiento ϵ-Decay, que combinan explotación determinista con exploración aleatoria; (ii) **UCB** (UCB1, UCB2, UCB1-Tuned), que racionalizan la exploración mediante límites de confianza superior sobre la incertidumbre de cada brazo; y (iii) **Ascenso de Gradiente** (Softmax y Preferencias), que aprenden una distribución de selección sobre las acciones. Los resultados muestran que no existe un algoritmo universalmente óptimo: UCB2 sobresale en entornos Normales, mientras que el Gradiente de Preferencias ($\alpha = 0.5$) domina en el régimen binario Bernoulli con un arrepentimiento acumulado prácticamente nulo (Regret ≈ 0).

Capítulo 2 — Entornos Complejos: Métodos Tabulares. Se formula el problema de control óptimo como un Proceso de Decisión de Markov (MDP) $\langle \mathcal{S}, \mathcal{A}, \mathcal{P}, \mathcal{R}, \gamma \rangle$ y se evalúa la familia de métodos tabulares model-free sobre el entorno `CliffWalking-v0`. Se implementan Monte Carlo On/Off-Policy, SARSA, Expected SARSA, Q-Learning, Double Q-Learning y variantes de n-pasos. Se diseña una arquitectura modular que desacopla el agente, el algoritmo (*Learner*) y el entorno (Gymnasium), garantizando reproducibilidad con $N = 10$ ejecuciones de 400 episodios ($\gamma = 0.95$, $\alpha = 0.5$). Los experimentos evidencian la superioridad de Expected SARSA (≈ -15 puntos de penalización) frente a Q-Learning (≈ -56) gracias a su robustez ante tasas de aprendizaje elevadas, y demuestran la ineficacia práctica de Monte Carlo en entornos con penalizaciones severas. Double Q-Learning resuelve el *Maximization Bias* a costa de una convergencia más lenta. El análisis concluye con el límite intrínseco de los métodos tabulares: la *maldición de la dimensionalidad* hace inviable la representación matricial $\mathbf{Q} \in \mathbb{R}^{|\mathcal{S}| \times |\mathcal{A}|}$ en espacios de estados continuos.

Capítulo 3 — Entornos Complejos: Métodos Aproximados. Para superar la barrera de escalabilidad, se sustituye la tabla $\mathbf{Q}$ por una función paramétrica $\hat{q}(\cdot\,; \boldsymbol{\theta})$ y se evalúan tres aproximadores de complejidad creciente sobre `CartPole-v1` ($\mathcal{S} \subseteq \mathbb{R}^4$) con $N = 10$ ejecuciones de 400 episodios: (i) **SARSA Semi-Gradiente** con Tile Coding ($d = 16\,384$ características binarias, $32\,768$ parámetros), cuya aproximación lineal limita la expresividad; (ii) **DQN** (red $4 \to 64 \to 64 \to 2$ con ReLU, Adam $\eta = 10^{-3}$, Experience Replay y red *target*), que combina la no linealidad neuronal con estabilización del proceso de Bellman; y (iii) **Double DQN**, extensión de DQN que desacopla selección y evaluación de la acción óptima para corregir el sesgo de sobreestimación. Los resultados (Promedio de recompensas en los últimos 50 episodios $\overline{R}_{[-50:]}$: SARSA 69, DQN 218, Double DQN 189) confirman que la capacidad expresiva del aproximador determina la calidad de la política, que el Experience Replay es condición crítica de estabilidad, y que la ventaja de Double DQN sobre DQN solo emerge en horizontes de entrenamiento superiores a $T_{\text{ep}} > 1\,000$ episodios.

En conjunto, los tres capítulos trazan una progresión coherente desde la exploración-explotación en un único estado hasta el control de sistemas dinámicos continuos mediante redes neuronales profundas, ilustrando los compromisos fundamentales del RL entre **capacidad expresiva**, **eficiencia muestral** y **estabilidad del aprendizaje**.

Índice general

1. Problema del Bandido de k-brazos **2**

1.1. Introducción 2

1.2. Desarrollo 2

1.3. Algoritmos 3

1.4. Evaluación y experimentos 4

1.5. Conclusiones 6

2. Entornos Complejos: Métodos Tabulares **8**

2.1. Introducción 8

2.2. Desarrollo 8

2.3. Algoritmos 9

2.4. Evaluación y experimentos 10

2.5. Conclusiones 13

3. Entornos Complejos: Métodos Aproximados **14**

3.1. Introducción 14

3.2. Desarrollo 14

3.3. Algoritmos 15

3.4. Evaluación y experimentos 17

3.5. Conclusiones 19

Capítulo 1

Problema del Bandido de k-brazos

1.1. Introducción

1.1.1. Descripción del problema y relevancia

El problema del bandido de k-brazos (o *Multi-Armed Bandit*) ilustra formalmente el dilema fundamental del Aprendizaje por Refuerzo: el balance iterativo entre la exploración y la explotación. Un agente computacional se enfrenta a un entorno estacionario y estocástico modelado como una máquina tragaperras con k palancas (brazos), cada una ocultando una distribución de probabilidad de recompensa subyacente diferente y desconocida *a priori*.

Con un tiempo límite o presupuesto de T interacciones, la meta del agente es maximizar su recompensa acumulada. La toma de su decisión es crítica: si confía únicamente en la palanca que le ha concedido mayor ganancia económica hasta ahora (explotación), cae en el riesgo de converger hacia una solución sub-óptima ignorando una palanca numéricamente superior. Si, por el contrario, invierte un esfuerzo desmedido en probar nuevas palancas aleatorias para descubrir el verdadero margen de beneficio (exploración), desperdicia iteraciones cobrando recompensas bajas con la consecuente penalización temporal.

1.1.2. Motivación del trabajo

La razón principal por la que estudiamos este problema es porque se parece a muchas decisiones prácticas que tomamos en el mundo real (como elegir en qué anuncio invertir dinero o qué tratamiento médico probar). Es un entorno "simple" porque las decisiones que tomamos ahora no cambian las opciones que tendremos en el futuro (cada tirada es independiente). Sin embargo, estudiarlo es clave para entender cómo funciona la inteligencia artificial cuando aprende a base de prueba y error.

Si logramos comprender cómo estos algoritmos básicos (ϵ-Greedy o UCB) interactúan con la duda de *"¿me quedo con lo que ya sé que funciona o arriesgo a probar algo nuevo?"*, habremos construido los pilares necesarios para entender a los verdaderos agentes robóticos complejos que resolverán problemas mucho más difíciles en el futuro.

1.1.3. Objetivos del informe

El desarrollo de esta primera fase trata de cumplir los siguientes objetivos técnicos: 1. Diseñar y programar un entorno modular en Python capaz de modelar un Bandido bajo tres tipologías de distribución de recompensa continuas y discretas: Normal, Binomial y Bernoulli. 2. Implementar desde cero tres familias clásicas de algoritmos de toma de decisiones: Explotación estocástica (ϵ-Greedy y sus variantes con decaimiento), Ascenso de Gradiente (Softmax y métricas de Preferencia) y Optmismo basado en incertidumbre (UCB-1, UCB-2 y UCB1-Tuned). 3. Evaluar dichas familias sometiéndolas empíricamente frente al entorno mediante $N = 500$ ejecuciones independientes simulando $T = 300$ pasos temporales. 4. Analizar la escalabilidad, la ratio de acierto de selección óptima y la mitigación del arrepentimiento acumulado (*Cumulative Regret*).

1.1.4. Organización del capítulo

Para facilitar su asimilación, este primer capítulo estructura su narrativa en el siguiente formato secuencial. En la sección de **Desarrollo** (1.2), se sentarán las bases teóricas definiendo las distribuciones de probabilidad a las que se enfrenta el bandido, adjuntando a la par un breve estado del arte de las influencias bibliográficas. Seguidamente, en la sección de **Algoritmos** (1.3), se expondrá la matemática formal y el pseudocódigo subyacente de cada política heurística testeada. Las asunciones y las simulaciones se materializan descriptivamente de la mano de gráficas de rendimiento en la sección referente a la **Evaluación/Experimentos** (1.4). Por último, los resultados son discutidos críticamente en las **Conclusiones** (1.5) para certificar el descubrimiento de los parámetros ganadores globales.

1.2. Desarrollo

1.2.1. Contexto y antecedentes

El dilema de exploración frente a explotación es una encrucijadas elemental dentro del Aprendizaje por Refuerzo. Mientras que en el aprendizaje supervisado un algoritmo recibe explícitamente la respuesta correcta a modo de etiqueta, en el aprendizaje evaluativo (característico del RL) un agente debe descubrir autónomamente qué acciones brindan la mayor ganancia probándolas. El modelo del *Multi-Armed Bandit* (Bandido Multibrazo) es el paradigma para aislar y estudiar estas propiedades evaluativas y de decisión en un entorno estacionario y no asociativo (es decir, donde el entorno no cambia de estado). Comprender la balanza entre arriesgar ganancias seguras presentes y el potencial de ganancias futuras desconocidas cimienta la matemática que los agentes complejos aplican cuando operan con los valores Q y variables de estado multidimensionales.

1.2.2. Definición formal del problema

El entorno estocástico se define bajo el espacio de recompensas discretas o continuas para un conjunto finito de acciones o brazos $a \in \mathcal{A} = \{1, 2, \ldots, k\}$.

El verdadero y desconocido "valor esperado" de elegir una acción a se denota habitualmente como el valor a maximizar $q_*(a)$, y está formulado como la esperanza matemática de la recompensa en un instante t:

$$q_*(a) = \mathbb{E}[R_t \mid A_t = a]$$

A lo largo del presente marco de validación, las recompensas R_t de cada brazo derivan su valor estocástico basándose en tres entornos implementados:

1. **Entorno de Recompensas Normales (Gausianas)**. La retribución estocástica de cada acción se muestrea de una distribución normal $R_t \sim \mathcal{N}(\mu_a, \sigma_a^2)$, donde la verdadera medida subyacente de recompensa está dominada por la siguiente función de densidad de probabilidad:

$$f(x \mid \mu_a, \sigma^2) = \frac{1}{\sqrt{2\pi\sigma^2}} e^{-\frac{(x-\mu_a)^2}{2\sigma^2}}$$

2. **Entorno de Recompensas Binomiales**. En este escenario, la activación del brazo a devuelve una recompensa entera discreta ($x \in \mathbb{N}$) que contabiliza la cantidad total de éxitos sobre n_a ensayos independientes. Es dirigido por la variable de masa probabilística $R_t \sim B(n_a, p_a)$:

$$P(R_t = x) = \binom{n_a}{x} p_a^x (1 - p_a)^{n_a - x}$$

3. **Entorno de Recompensas de Bernoulli**. Un escenario más sencillo donde el resultado de tirar de la palanca solo puede ser un éxito o un fracaso absoluto ($R_t \in \{0, 1\}$). Modela la variable $R_t \sim Bernoulli(p_a)$ en donde el valor promedio de iteración a estabilizar es exactamente la probabilidad de éxito intrínseca al brazo evaluado ($q_*(a) = p_a$):

$$P(R_t) = p_a^{R_t} (1 - p_a)^{1 - R_t}$$

1.2.3. Enfoque y justificación metodológica

Para abordar el problema estocástico, se ha abstraído un *framework* orientado a objetos que desacopla completamente el "Entorno" del "Agente". Esto permite insertar módulos de políticas que basan su decisión algorítmica interactuando con las mismas estructuras pre-configuradas. El presente trabajo se caracteriza por implementar la comparativa simultánea y simétrica del estado del arte (ϵ-*Greedy* frente al Límite de Confianza *UCB* y el cálculo heurístico *Softmax*) bajo un volumen estadístico ($N = 500$ iteraciones promediadas), garantizando un estudio numérico de optimización con mitigación del *Arrepentimiento Acumulado* en ecosistemas tanto con recompensas continuas como discretas.

1.2.4. Trabajos relacionados

El planteamiento algorítmico, su formulación paramétrica y las heurísticas estandarizadas analizadas en este proyecto se fundamentan y se derivan de la literatura existente en el campo del Aprendizaje por Refuerzo:

- **Métodos Epsilon y Ascenso de Gradiente (Sutton y Barto, 2018):** El diseño de las estrategias ϵ-Greedy y de los métodos probabilísticos como el *Ascenso de Gradiente* se ha construido siguiendo las reglas y fundamentos explicados en el libro de referencia "Reinforcement Learning: An Introduction" [1].

- **Manejo de la Incertidumbre con UCB (Auer et al., 2002):** Para programar los algoritmos UCB (Upper Confidence Bound) de forma robusta y entender cómo utilizan sus fórmulas matemáticas para explorar las opciones menos conocidas, nos hemos apoyado en el estudio "Finite-time Analysis of the Multiarmed Bandit Problem" [2].

1.3. Algoritmos

Para resolver el problema del bandido de k-brazos hemos implementado tres enfoques distintos: métodos que combinan explotar lo conocido con explorar al azar, algoritmos que premian probar opciones inciertas para dejar de dudar, y tácticas que eligen las acciones basándose en preferencias relativas en lugar de en ganancias directas. A continuación se justifican sus enfoques y se detalla el funcionamiento lógico de los pilares principales de cada familia algorítmica estudiada.

1.3.1. Familia 1: Lógicas de exploración-explotación (ϵ-Greedy)

El algoritmo ϵ-Greedy (Algorithm 1) es el punto de partida más clásico en el Aprendizaje por Refuerzo. Su funcionamiento es muy sencillo y rápido de calcular: la mayor parte del tiempo (con probabilidad $1 - \epsilon$) el agente elige la opción que hasta ahora le ha dado más beneficios, pero de vez en cuando (con una pequeña probabilidad ϵ) elige una opción al azar para explorar. A partir de aquí nace una mejora llamada ϵ-Decay, que soluciona el problema de explorar para siempre: va reduciendo poco a poco la probabilidad de explorar (multiplicándola por un factor λ), de modo que, a medida que el agente aprende, se vuelve más seguro y se centra en asegurar las ganancias.

Algorithm 1 Algoritmo ϵ-Greedy para K-Brazos

Require: $\epsilon \in [0, 1]$, Total de acciones K
Inicializar $Q(a) \leftarrow 0$ para todo $a = 1, \ldots, K$
Inicializar $N(a) \leftarrow 0$ para todo $a = 1, \ldots, K$
for cada paso $t = 1, 2, \ldots, T$ **do**
  $p \leftarrow$ Valor aleatorio de Distribución $\mathcal{U}(0, 1)$
  if $p < \epsilon$ **then**
    $A_t \leftarrow$ Acción aleatoria uniforme en $\{1, \ldots, K\}$
  else
    $A_t \leftarrow \arg\max_a Q(a)$ ▷ Ruptura de empates al azar
  end if
  Ejecutar A_t, observar recomensa R_t
  $N(A_t) \leftarrow N(A_t) + 1$
  $Q(A_t) \leftarrow Q(A_t) + \frac{1}{N(A_t)}\left[R_t - Q(A_t)\right]$
end for

1.3.2. Familia 2: Optimismo ante la incertidumbre (UCB)

A diferencia de explorar siempre al azar, los métodos de *Upper Confidence Bound* (UCB) son curiosos de forma mucho más inteligente. UCB calcula cuánta confianza tiene en lo que ya sabe de cada palanca. Si una opción se ha probado muy poco, su nivel de incertidumbre es alto, así que el algoritmo decide elegirla para "salir de dudas". Esto convierte a UCB en un método robusto y bueno para no perder el tiempo. En este trabajo hemos estudiado el algoritmo básico UCB1 (que usa un número c para controlar de forma fija esa curiosidad), el modelo UCB2 (que optimiza el proceso probando las opciones en "ráfagas" o bloques cada vez más largos en lugar de recalcular a cada paso), y la variante avanzada UCB1-Tuned (que se ajusta a sí misma midiendo la variabilidad de los premios que recibe). Mostramos el pseudocódigo de UCB2 en Algorithm 2.

Algorithm 2 Algoritmo UCB2

Require: Constante exploratoria $\alpha \in (0, 1)$
Inicializar contador de épocas por brazo: para cada a, $r_a \leftarrow 0$
Iniciar explorando todos los brazos una vez, actualizando su recompensa promedio $Q(a)$
for paso de tiempo $t = K + 1, \ldots$ **do**
  $A_t \leftarrow \arg\max_a \left[Q(a) + \sqrt{\frac{(1+\alpha)\ln(e \cdot t/\tau(r_a))}{2\tau(r_a)}} \right]$ (donde $\tau(r) = \lceil (1+\alpha)^r \rceil$)
  Determinar longitud de la ráfaga: $\Delta \leftarrow \tau(r_{A_t} + 1) - \tau(r_{A_t})$
  for iteraciones $i = 1$ hasta Δ **do**
    Ejecutar A_t, percibir recompensa R
    $N(A_t) \leftarrow N(A_t) + 1$
    $Q(A_t) \leftarrow Q(A_t) + \frac{1}{N(A_t)} \left[R - Q(A_t) \right]$
  end for
  Actualizar época del brazo: $r_{A_t} \leftarrow r_{A_t} + 1$
end for

1.3.3. Familia 3: Ascenso de gradiente estocástico (Softmax y Preferencias)

Si los algoritmos solo se fijan en los puntos exactos de cada opción (Q), pueden tener problemas para decidir si los premios son muy parecidos (por ejemplo, ganar +85 o +84). Los métodos basados en el Gradiente lo resuelven usando "Preferencias" (H_a). En vez de memorizar valores numéricos rígidos, simplemente aprenden qué opción prefieren en relación al resto y deciden usando porcentajes (con la Ley Exponencial Suave o Gibbs). Es decir, estos métodos emplean una distribución de probabilidad para decidir la siguiente acción a realizar. El pseudocódigo del algoritmo de Gradiente de Preferencias se muestra en Algorithm 3.

Algorithm 3 Algoritmo de Gradiente de Preferencias

Require: Tasa de aprendizaje $\alpha > 0$
Inicializar preferencias $H(a) \leftarrow 0 \ \forall a$
Inicializar recompensa promedio de validación $\bar{R} \leftarrow 0$
for $t = 1, \ldots, T$ **do**
  Calcular probabilidades de la curva Softmax:
$P(a) \leftarrow \frac{\exp(H(a))}{\sum_{b=1}^{K} \exp(H(b))} \quad \forall a$
  $A_t \leftarrow$ Valor aleatorio de distribución de densidad $P(\cdot)$
  Obtener recompensa R_t
  Actualizar la base empírica global $\bar{R} \leftarrow \bar{R} + \frac{1}{t}(R_t - \bar{R})$
  for all $a \in \{1, \ldots, K\}$ **do**
    if $a = A_t$ **then**
      $H(a) \leftarrow H(a) + \alpha(R_t - \bar{R})(1 - P(a))$
    else
      $H(a) \leftarrow H(a) - \alpha(R_t - \bar{R})P(a)$
    end if
  end for
end for

1.4. Evaluación y experimentos

1.4.1. Configuración experimental

Para garantizar la validez científica y la total reproducibilidad de los resultados a lo largo del estudio de las tres familias de distribuciones (Normal, Binomial y Bernoulli), el entorno de pruebas fue estandarizado con los siguientes hiperparámetros de simulación global: - **Ejecuciones (*Runs*):** $N = 500$ pruebas independientes estocásticas. Las curvas finales presentadas corresponden a la matriz promediada probabilísticamente sobre este número total de semillas. - **Horizonte de Simulador (*Steps*):** $T = 300$ pasos temporales de decisión por ejecución. - **Semilla Base Estática:** Se inyectó globalmente una semilla matemática (aleatoria constante, habitualmente `np.random.seed(1024)`) para inicializar la topología generatriz del bandido de forma idéntica ante la comparativa de los tres algoritmos.

Para llevar a cabo las simulaciones, se definieron para cada algoritmo las parrillas de hiperparámetros que se muestran en la Tabla 1.1.

1.4.2. Métricas de desempeño

La evaluación del rendimiento integral de los agentes propuestos se analizó multidimensionalmente observando su desempeño iterativo bajo 4 prismas métricos: **1. Recompensa Promedio Continua (*Average Reward*):** Traza iterativa de la estimación $\bar{R}_t$; refleja la capacidad del agente de percibir y adaptar ganancia bruta promedio. **2. Ratio de Selección del Brazo Óptimo (*% Optimal Action*):** Discrimina de forma binaria el acierto o el fallo explícito al localizar la mejor distribución subyacente $\arg\max q_*(a)$. Es decir, índica el porcentaje de veces que se ha seleccionado el brazo óptimo en cada paso. **3. Arrepentimiento Acumulado (*Cumulative Regret*):** Mide la acumulación de pérdida teórica u "oportunidad perdida" a nivel temporal de decantarse por brazos subóptimos. Se busca priorizar gráficas logarítmicas y rápidamente estabilizadas (planas). **4. Histogramas Distribucionales:** Distribuciones de las recompensas obtenidas con cada brazo, así como el número de veces que cada brazo ha sido seleccionado (*Arm Statistics*).

1.4.3. Análisis de resultados

La convergencia de rendimiento obtenida tras los experimentos numéricos extraídos de `epsilon_greedy.ipynb`, `ascenso_gradiente.ipynb` y `UCB_algorithms.ipynb` ha po-

Tabla 1.1: Hiperparámetros definidos y comparados por familia algorítmica.

Familia Algorítmica	**Configuración / Algoritmo**	**Hiperparámetros Evaluados**
Exploración-Explotación	ϵ-Greedy	Tasa de exploración $\epsilon \in \{0.0, 0.01, 0.1\}$
Exploración-Explotación	ϵ-Decay	Tasa de decaimiento $\lambda \in \{0.8, 0.99, 0.9999\}$
Ascenso de Gradiente	Softmax	Temperatura $\tau \in \{0.5, 1.0, 2.0\}$
Ascenso de Gradiente	Gradiente de Preferencias	Tasa de aprendizaje $\alpha \in \{0.5, 1.0, 2.0\}$
Límite Sup. Condianza	UCB1	Constante de exploración $c \in \{0.5, 1.0, 1.5\}$
Límite Sup. Condianza	UCB2	Expansión de confianza temporal $\alpha \in \{0.25, 0.5, 0.75\}$
Límite Sup. Condianza	UCB1-Tuned	Ninguno (Varianza de la Recompensa empírica)

sibilitado la categorización de parámetros óptimos para las tres ramas del estado del arte, englobados en la Tabla 1.2.

Resultados familia ϵ-Greedy

- **Decaimiento (ϵ-Decay):** Aunque en teoría debería ser mejor, en la práctica el decaimiento no ha supuesto ninguna ventaja clara en los entornos *Normal* y *Binomial*. De hecho, ha funcionado igual o ligeramente **peor** que el modelo básico (ϵ-Greedy puro). De todos los ajustes probados expuestos en la Figura 1.1, reducir la probabilidad de explorar progresivamente con un multiplicador $\lambda = 0.99$ fue lo más equilibrado. En cambio, si la exploración se reduce muy despacio ($\lambda = 0.9999$), el algoritmo pierde demasiado tiempo escogiendo opciones al azar, acumulando muchas pérdidas.
- **Bernoulli**: En este caso particular, hemos experimentado un problema grave (visible en la Figura 1.2): si frenamos la caída de exploración demasiado pronto (fijando un mínimo inamovible de $\epsilon_{min} = 0.001$), el algoritmo deja de aprender de golpe. Esto provoca que se estanque prematuramente y no logre igualar la enorme tasa de aciertos que sí consiguen otros competidores, como el propio ϵ-Greedy. Notamos que el pico que se aprecia al principio de la gráfica se debe a una fase de inicialización en la que el agente prueba cada brazo una vez.

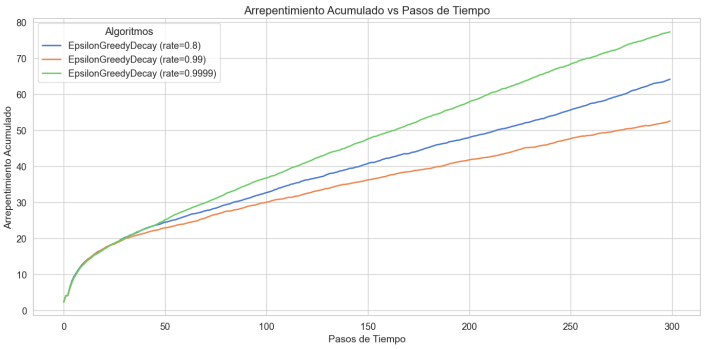


Figura 1.1: Curva de Arrepentimiento Acumulado de ϵ-Decay (entornos Normales).

Resultados familia de Ascenso de gradiente

- **Softmax frente a los entornos:** Mientras que el método Softmax se defiende de forma aceptable en entornos donde los premios varían de forma continua (distribución *Normal*), fracasa completamente cuando los premios son de "todo o nada" (*Bernoulli*). Como se observa detalladamente en la Figura 1.3, en este último caso el algoritmo se atasca rápidamente, eligiendo la mejor opción apenas un 20 % de las veces y acumulando muchísimas pérdidas a lo largo del tiempo.

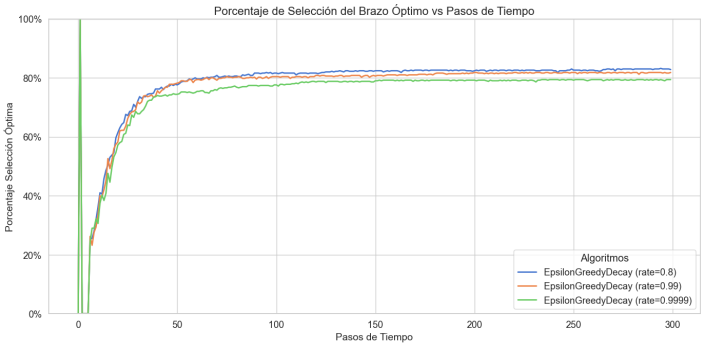


Figura 1.2: Estancamiento del Ratio de Selección del Brazo Óptimo de ϵ-Decay ($\epsilon_{min} = 0.001$) en Bernoulli.

- **Gradiente de Preferencias vs Softmax:** Este ha sido uno de los grandes descubrimientos de las pruebas. El *Gradiente de Preferencias*, al guiarse por "cuánto le gusta" una opción (porcentajes de preferencia) en lugar de intentar memorizar valores exactos de puntos, **ha superado con mucha claridad a Softmax en todos los escenarios**. Su éxito es espectacular en el entorno *Bernoulli*, donde consigue elegir la mejor palanca casi el 100 % de las veces desde muy temprano. La Figura 1.4 constata cómo esto hace que sus pérdidas (arrepentimiento) se queden en unos pobres 3 puntos tras 300 intentos (frente a los 70 puntos perdidos por Softmax).

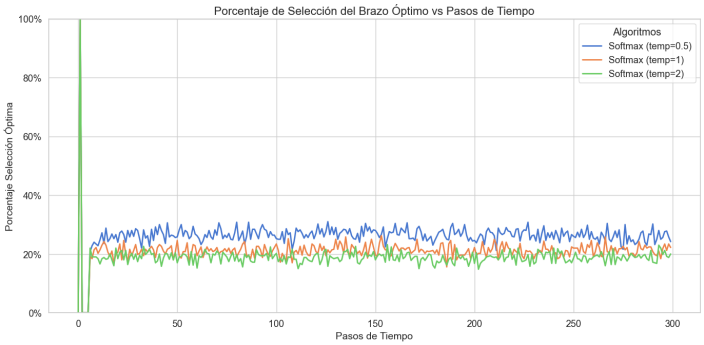


Figura 1.3: Ratio de Selección del Brazo Óptimo de Softmax en el entorno Bernoulli.

Resultados familia UCB

- **UCB1 vs. UCB2:** El modelo clásico UCB1 aprende bastante bien en casi todas las situaciones probadas, sobre todo cuando su nivel de confianza inicial está fijado

Tabla 1.2: Parámetros con mayor rendimiento cruzados por distribución.

Familia	**Mejor Hiperparámetro**	**Análisis Crítico**
ϵ-Greedy	$\epsilon = 0.1$	Globalmente superior a versiones estáticas conservadoras ($\epsilon = 0.01$).
Decaimento	$\lambda = 0.99$	Combina convergencia exponencial de exploración. Cae veloz frente a entornos como Bernoulli si su suelo (ϵ_{min}) es severo.
Softmax	Temperatura $\tau = 0.5$	Curva modesta en continuas; catastrófico (20 % acierto) en dominios Bernoulli.
P. Gradiente	Ratio de Aprendizaje $\alpha = 0.5$	La revelación. Su matriz de distancias absolutas mitiga por completo entornos acotados y discretos (Bernoulli).
UCB	$c = 1.0$ (o $c = 0.5$ Bernoulli)	Curvas de convergencia teórica muy seguras y logarítmicas. Rinde muy bien para incertidumbres Gausianas.
UCB1-Tuned	Ninguno (No paramétrico)	Defensa inquebrantable en todas las distribuciones; el entorno escala su nivel de exploración por la varianza natamente.

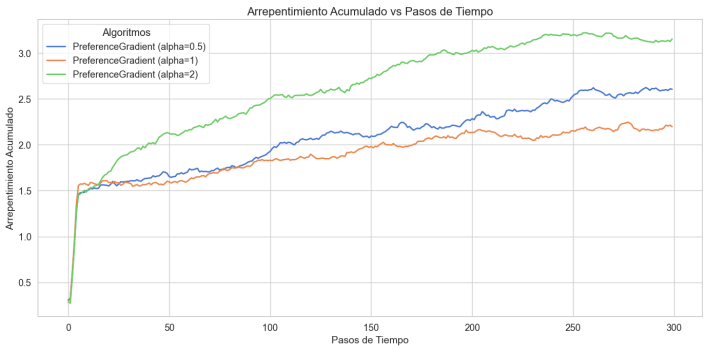


Figura 1.4: Regret planificado demostrando el éxito absoluto de Gradiente de Preferencias en Bernoulli.

Figura 1.5: Fracaso rotundo y de penalización lineal de UCB2 en entornos discretos Bernoulli.

en $c = 1.0$ (para los modelos *Normal* y *Binomial*). Sin embargo, en el juego de premios binarios (*Bernoulli*) le va mejor si arranca más precavido con un $c = 0.5$. Su hermano mayor, **UCB2**, muestra dos caras opuestas: **falla estrepitosamente en los entornos de Binomial y Bernoulli**, donde se atasca eligiendo constantemente la peor opción sin aprender casi nada (véase la escalada incontrolable de la Figura 1.5), pero a cambio resulta ser rapidísimo y el mejor de todos cuando los premios son más naturales y variados (entornos *Normales*), como se observa en la Figura 1.6, donde vemos un arrepentimiento prácticamente horizontal.

- **UCB1-Tuned**: Esta versión con "piloto automático" consiguió igualar algunos de los buenos resultados de UCB1 sin necesidad de configurar ningún parámetro a mano. Aún así, sigue pasándolo bastante mal cuando las reglas del juego son drásticas (como en el entorno *Bernoulli*), donde, tal y como se aprecia en la Figura 1.7, necesita atascarse al menos 100 intentos seguidos antes de lograr darse cuenta de cuál es el brazo ganador.

Para ver todo esto de forma más clara, la mayoría de las gráficas mostradas han sido las de arrepentimiento acumulado puesto que, tal y como descubrimos al analizar a fondo las simulaciones, **esta gráfica resulta ser siempre la más útil y fácil de entender**. Esto se debe a que, a diferencia de otras medidas, en el Arrepentimiento se puede distinguir a simple vista si un algoritmo sigue aprendiendo y reduciendo sus pérdidas o si ya se ha estancado por completo.

1.5. Conclusiones

1.5.1. Resumen de principales resultados

Tras probar a fondo todos los algoritmos en el problema del bandido de k-brazos, sacamos una conclusión clara: no existe un algoritmo perfecto para todo. La mejor estrategia siempre dependerá del tipo de entorno y de cómo se comporten los premios.

- En entornos donde las recompensas varían de forma continua y lógica (como en la distribución *Normal*), métodos como **UCB1** y especialmente **UCB2** aprenden rapidísimo al elegir de forma inteligente qué opciones explorar. Sin embargo, **solo UCB1** logra mantener ese buen nivel en entornos algo más rígidos como el *Binomial*, ya que UCB2 se atasca por completo.
- Cuando nos enfrentamos a escenarios drásticos de "todo o nada" (recompensas *Bernoulli*), casi todos los algoritmos anteriores fallan de forma estrepitosa y acumulan pérdidas. En estos casos tan extremos, el ganador es el

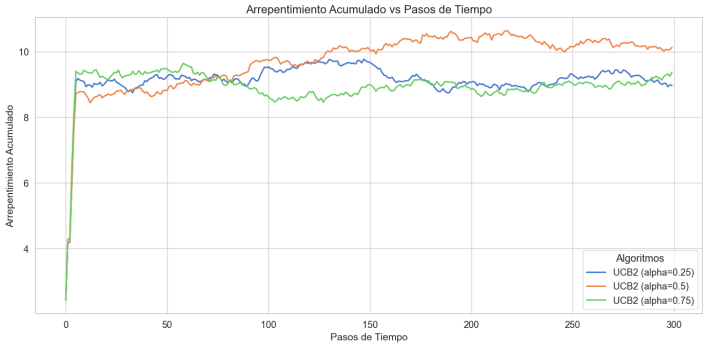


Figura 1.6: Éxito absoluto de UCB2 en entornos de distribución normal.

Figura 1.7: Estancamiento inicial pronunicado de UCB1-Tuned frente al engaño paramétrico Bernoulli.

Gradiente de Preferencias. En vez de obsesionarse con calcular exactamente cuántos puntos da cada opción, este modelo se limita a comparar qué tan buena es una palanca respecto de las demás; gracias a esto, logra dominar el juego y no equivocarse casi nunca ($Regret \approx 0$).

- Para situaciones variadas o cuando no tenemos mucha potencia de cálculo, recurrir a los clásicos métodos semialeatorios (como ϵ**-Decay** o directamente un ϵ fijo de 0.1) resulta ser una estrategia buena.

1.5.2. Limitaciones del estudio

La limitación principal de los resultados reportados en este documento es que el juego asume que las reglas no cambian, la propiedad de **estacionalidad** del modelo K-Brazos abstraído. En el mundo real, la característica estocástica y las medias de recompensa (μ) de una máquina o variable evolucionan y mutan con el paso del tiempo. Si trasladamos estos algoritmos a la vida real, estos algoritmos podrían fracasar estrepitosamente porque se obcecarían en descartar opciones de escenarios que de repente podrían volverse muy rentables.

Además, este marco experimentacional excluye variables de "contexto" (variables climáticas, visuales o previas a la decisión) de cada tirada o instante t, tratando a las palancas bajo el paradigma probabilístico absoluto e idéntico en cada *step*.

1.5.3. Impacto y reflexión en el Aprendizaje por Refuerzo

Estudiar a fondo estas tres familias de algoritmos del bandido (apostar a lo seguro, investigar las opciones menos conocidas, o guiarse por las preferencias) no es para nada un ejercicio anticuado o puramente teórico. Al contrario: el concepto del *Arrepentimiento* y cómo minimizarlo son los cimientos más básicos que le dicen a cualquier Inteligencia Artificial cómo debe tomar sus decisiones.

Estas fórmulas e ideas matemáticas conforman el motor interno inicial de decisiones de cualquier agente moderno y complejo en el mundo real (como los sistemas de *Deep Reinforcement Learning* de las IA generativas o de los coches autónomos).

1.5.4. Líneas futuras de estudio

El siguiente paso natural para avanzar en este campo sería crear simulaciones de **Bandidos No Estacionarios (*Non-stationary Bandits*)**, es decir, enseñar a la Inteligencia Artificial a jugar en entornos donde las reglas y los premios van cambiando con el tiempo. Para lograrlo, habría que dotar a los algoritmos con capacidades de adaptación "olvidadizas": mecanismos matemáticos que les permitan ignorar lecciones antiguas que ya no sirven y centrarse en lo que está ocurriendo ahora mismo.

Más adelante, el objetivo final sería evolucionar hacia **Bandidos Contextuales (*Contextual Bandits*)**. En este escenario, la IA no jugaría a ciegas, sino que recibiría "pistas" del entorno antes de tener que elegir una opción (por ejemplo, saber qué tiempo hace antes de recomendar un tipo de ropa). Esto abre directamente las puertas hacia sistemas de inteligencia artificial mucho más completos y complejos que los que hemos visto hasta ahora.

Capítulo 2

Entornos Complejos: Métodos Tabulares

2.1. Introducción

2.1.1. Descripción del problema y relevancia

El Aprendizaje por Refuerzo (RL) en entornos complejos modela problemas secuenciales de toma de decisiones donde un agente interactúa de forma activa con un ecosistema dinámico temporal. A diferencia de escenarios estocásticos simples y puramente exploratorios (como el bandido multibrazo), en estos entornos el estado del sistema cambia iterativamente como consecuencia directa de las acciones ejecutadas, formalizándose matemáticamente a través del Proceso de Decisión de Markov (MDP). La resolución de estos problemas mediante métodos tabulares es de suma relevancia, dado que cimienta las bases analíticas del control óptimo para escenarios discretos y finitos, trazando el camino fundamental hacia algoritmos de inteligencia artificial más robustos e independientes del problema.

2.1.2. Motivación del trabajo

La motivación de este proyecto radica en la necesidad de diseñar agentes autónomos capaces de converger hacia políticas de control óptimas "a ciegas", presumiendo una ignorancia total respecto a las dinámicas probabilísticas de transición ($\mathcal{P}$) y del modelo subyacente de recompensas ($\mathcal{R}$). En los problemas físicos realistas, aplicar técnicas basadas en Programación Dinámica (PD) perfecta de Bellman para resolver estos MDPs es computacionalmente inútil por la falta del modelo de entorno. Analizar cómo interactúan empíricamente familias algorítmicas de *bootstrapping* de uno o más pasos (Diferencias Temporales) y evaluaciones completas de episodios (Monte Carlo) resulta imprescindible para interiorizar el funcionamiento de los métodos tabulares y comparar su efectividad en un mismo entorno.

2.1.3. Objetivos del informe

Los objetivos que se persiguen con este desarrollo algorítmico son los siguientes:

1. Diseñar e implementar computacionalmente desde cero el estado del arte de las familias algorítmicas de control discreto de aprendizaje tabular iterativo (Monte Carlo, SARSA, Q-learning, Expected SARSA, Doble Q-Learning y variantes de n-pasos).
2. Construir una arquitectura evaluativa agnóstica soportada en la librería de simulación física estandarizada Gymnasium, empleando mapas orientados al estrés y sensibilidad de castigos de agentes (*CliffWalking-v0*).
3. Someter los modelos a una batería de pruebas, visualizando analíticamente sus fluctuaciones de convergencia en cuanto a las recompensas obtenidas, la longitud promedio de los episodios y los errores de aprendizaje, fundamentalmente.
4. Demostrar empíricamente las fronteras matemáticas y la "maldición de la dimensionalidad" intrínsecas a las tablas espaciales determinísticas $\mathbf{Q} \in \mathbb{R}^{|\mathcal{S}|\times|\mathcal{A}|}$.

2.1.4. Organización del capítulo

Este capítulo se estructura sistemáticamente acorde a la siguiente distribución: la Sección 3.2 aborda el marco teórico, desarrollo formal del problema y la vinculación contextual con trabajos literarios relacionados en materia estocástica tabular. En la Sección 3.3, se exhiben en profundidad los paradigmas algorítmicos y sus reglas de actualización. Posteriormente, la Sección 3.4 documenta de manera meticulosa la configuración experimental y disecciona gráficamente las comparativas de resultados cosechadas al llevar a cabo las rutinas propuestas. Para finalizar, la Sección 3.5 cristaliza las reflexiones concluyentes derivadas del desempeño, sus limitaciones y las sendas futuras de adaptación hacia aproximadores funcionales con gradientes.

2.2. Desarrollo

2.2.1. Contexto y antecedentes

Para asimilar el marco tabular implementado es necesario comprender el desacoplo matricial de la función de valor asintótico. Los métodos tabulares descartan la regresión funcional asumiendo un precondicionamiento dimensional finito, esto es, en el que la calidad del par estado-acción se aloja en ubicaciones explícitas de una matriz $\mathbf{Q} \in \mathbb{R}^{|\mathcal{S}|\times|\mathcal{A}|}$. Este proyecto subyace bajo la transición directa hereditaria del problema del bandido multibrazo estático hacia el modelado temporal dinámico. Aquel primer entorno forjó empíricamente el dilema subyacente de la *Exploración frente a Explotación*, asimilando políticas iniciales de decisión como el esquema ϵ-Greedy:

$$\pi(a \mid s) = \begin{cases} 1 - \epsilon + \frac{\epsilon}{|\mathcal{A}|} & \text{si } a = \arg\max_{a'} Q(s, a') \\ \frac{\epsilon}{|\mathcal{A}|} & \text{en caso contrario} \end{cases}$$

2.2.2. Definición formal del problema

El núcleo central abordado consiste en la optimización de políticas de control en escenarios bajo incertidumbre, modelado formalmente de forma canónica mediante un Proceso de Decisión de Markov (MDP). Un MDP finito queda univocamente estipulado por la tupla matemática $\langle \mathcal{S}, \mathcal{A}, \mathcal{P}, \mathcal{R}, \gamma \rangle$, donde:

- $\mathcal{S}$ define el conjunto discreto y finito de estados.
- $\mathcal{A}$ define el conjunto discreto de acciones permitidas.
- $\mathcal{P}(s' \mid s, a)$ representa formalmente la probabilidad de transición hacia un estado subsecuente $s' \in \mathcal{S}$ habiendo tomado la acción $a \in \mathcal{A}$ en el estado de partida $s \in \mathcal{S}$.
- $\mathcal{R}(s, a, s')$ representa la función formal de la recompensa inmediata obtenida en dicha transición.
- $\gamma \in [0, 1]$ establece analíticamente el factor de descuento temporal que pondera la relevancia de los retornos futuros.

El desafío resuelto se contextualiza bajo la condición del control óptimo empírico: el agente carece de un modelo dinámico *white-box*, ignorando totalmente $\mathcal{P}$ y $\mathcal{R}$. A diferencia de las metodologías de Programación Dinámica que asumen la resolución exacta sobre las Ecuaciones de Bellman, este trabajo exige destilar una matriz de política óptima π_* operando "a ciegas" sobre las recompensas interactivas puras que brinda la librería estandarizada *Gymnasium*. En nuestro enfoque de aplicación específico, el espacio queda materializado sobre el grid topográfico 4x12 *CliffWalking*, donde el objetivo es encontrar la ruta ideal que logre alcanzar la casilla objetivo sin caer a un precipicio que penalizaría al agente con una recompensa de 100 puntos negativos.

2.2.3. Explicación de los métodos utilizados y enfoques existentes

La aproximación técnica empleada pivota sobre la disección de dos grandes familias algorítmicas canónicas en el ecosistema tabular de RL:

1. **Iteración Exhaustiva Episódica (Monte Carlo):** El agente juega una partida completa desde el principio hasta alcanzar un estado final (victoria o derrota), acumulando la suma de recompensas total obtenida (retornos G_t). Solo cuando el episodio termina, el algoritmo revisa la partida jugada y actualiza su matriz $\mathbf{Q}$ promediando los resultados de sus acciones. Aunque matemáticamente este método garantiza encontrar la solución óptima a largo plazo sin sesgos, presenta el gran inconveniente de ser muy inestable (alta varianza) en entornos donde un error mínimo conlleva un castigo catastrófico.
2. **Diferencias Temporales o *Bootstrapping* (TD):** Estos algoritmos no necesitan esperar a que finalice la partida completa para aprender. En lugar de ello, corrigen su memoria paso a paso apoyándose en sus propias intuiciones sobre el futuro inmediato (lo que se conoce como *bootstrapping*). Valoran cada acción tomada actualizando sus creencias basándose en la predicción de lo que ocurrirá en el siguiente estado, aprendiendo sobre la marcha. Estos métodos constituyen el núcleo motriz del proyecto, englobando implementaciones como Q-learning, SARSA y Double Q-learning. Además, son generalizables a versiones que actualizan la tabla $\mathbf{Q}$ cada n pasos, en lugar de en cada iteración, como la variante de *n-step* SARSA.

2.2.4. Justificación: Diferenciación y mejora frente a enfoques previos

El enfoque de este trabajo representa una mejora arquitectónica frente a otras implementaciones básicas. Habitualmente, el código del entorno (el mapa) y el algoritmo de aprendizaje (el cerebro) suelen estar entrelazados de forma inflexible. En nuestro proyecto, hemos construido una arquitectura modular donde el Agente, el Algoritmo (Learner) y el Entorno (Gymnasium) están totalmente separados. Gracias a este diseño, podemos evaluar exactamente el mismo modelo matemático tanto en el entorno del acantilado (*CliffWalking*) como en el del lago helado (*FrozenLake*) sin reescribir su lógica interna. Además, nuestra evaluación va un paso más allá de la simple recolección de puntos: analizamos específicamente cómo cada algoritmo gestiona el riesgo de caer en trampas fatales frente a su ambición ciega por encontrar la meta rápida.

2.2.5. Trabajos relacionados

La formalización analítica de los procesos de decisión y sus métodos tabulares subyacentes se apoya sobre los axiomas establecidos en la literatura por C. Watkins de Cambridge, quien introdujo el algoritmo empírico desacoplado de *Q-learning* [3], probando que una actualización puramente *Off-Policy* convergía formalmente hacia la ruta iterativa de la matriz óptima $\mathbf{Q}_*$ asumiendo visitaciones matemáticas al infinito. La familia *On-Policy* análoga descrita formalmente bajo el acrónimo SARSA (*State-Action-Reward-State-Action*) fue introducida poco después por los autores Rummery y Niranjan en [4].

La evolución conceptual de los métodos clásicos tabulares, en especial el fenómeno estadístico del *Maximization Bias* propio de heurísticas puramente deterministas orientadas al $\max_a$, propició directamente la publicación de Hado van Hasselt en NeurIPS [5], forjando por primera vez la escisión del predictor en el sistema denominado *Double Q-Learning*, que emplea dos tablas de valores Q simultáneamente. Todo este marco conceptual, que abarca desde el método de Monte Carlo hasta el aprendizaje temporal de n-pasos, se encuentra recopilado y unificado en el texto de referencia clásico de Sutton y Barto [1], el cual sirve como base metodológica principal para el desarrollo de este trabajo.

2.3. Algoritmos

El esqueleto algorítmico fundamental que es utilizado en todos los métodos de aprendizaje se halla englobado en la clase `Agent`. Ésta representa a un agente inteligente que interactúa con el entorno, escogiendo acciones en base a su política personal, transitando de un estado a otro, recibiendo recompensas y recopilando estadísticas. Una versión simplificada del funcionamiento de éste se puede ver en el Algoritmo 4.

En el pseudocódigo del Agente Inteligente observamos que todos aquellos métodos encargados de actualizar la tabla Q y, por ende, mejorar la política del agente, son relegados al algoritmo de aprendizaje L. Esto significa que, en función

Algorithm 4 Agente inteligente

Require: Entorno E, método de aprendizaje L, política de comportamiento π, número de episodios N, número de ejecuciones R.

```
for ejecución r = 1, ..., R do
    L.reset()    ▷ Resetear el algoritmo de aprendizaje en cada ejecución
    for episodio e = 1, ..., N do
        S ← E.reset()
        L.startEpisode()    ▷ Inicializar el episodio
        while e no termine do
            A ← seleccionar acción en base a π y S
            S', R ← E.step(A)
            L.step(S, A, R, S')    ▷ Paso del algoritmo de aprendizaje
            S ← S'
        end while
        L.endEpisode()    ▷ Finalizar el episodio
    end for
end for
```

del algoritmo de aprendizaje que queramos usar, será preciso redefinir estos métodos para que se ajusten a las bases teóricas de cada procedimiento.

Tal y como se ha mencionado en la Sección 2.2.3, existen dos paradigmas fundamentales dentro del conjunto de métodos tabulares: los métodos de Monte Carlo, que esperan a finalizar cada episodio para actualizar la tabla **Q**, y los métodos de Diferencias Temporales, que actualizan su tabla de valores en cada paso (o, en general, cada n pasos). Existe, además, otra clasificación que separa los métodos en función de si utilizan la misma política que el agente (métodos *On Policy*) o bien una política diferente (métodos *Off Policy*). Dada la gran cantidad de algoritmos diferentes, en este apartado nos limitamos a mostrar un algoritmo de Monte Carlo On Policy y otro de Diferencias Temporales Off Policy (Q-Learning). El lector interesado puede consultar el resto de algoritmos implementados en el módulo `learners` del código fuente.

Algorithm 5 Monte Carlo On Policy

Require: Episodio E, booleano que indica si es de primera visita V, tasa de descuento $\gamma \in [0, 1]$

```
G ← 0
E' ← copia de los (S, A) de E
for all (S, A, R) ∈ E.reverse() do
    G ← R + γG
    E'.pop()
    if V and (S, A) aparece en E' then
        Pasar a la siguiente iteración
    end if
    Visits(S, A) ← Visits(S, A) + 1
    α ← 1/Visits(S, A)    ▷ Actualización incremental
    Q(S, A) ← Q(S, A) + α · [G − Q(S, A)]
end for
```

En cuanto al método de Monte Carlo On Policy (Algoritmo 5), únicamente se muestra el pseudocódigo de la función endEpisode() puesto que es ahí donde se actualizan los valores Q, una vez el episodio ha concluido. Notamos que el algoritmo mostrado unifica las técnicas conocidas como Monte Carlo de primera visita, el cual solo tiene en cuenta el retorno de una pareja (S, A) la primera vez que aparece en el episodio, y Monte Carlo de todas las visitas, que calcula la media de todos los retornos observados para cada par (S, A).

Algorithm 6 Q-Learning (TD Off Policy)

Require: Tasa de aprendizaje $\alpha \in (0, 1]$, tasa de descuento $\gamma \in [0, 1]$, estado S, acción A, recompensa R, siguiente estado S'.

```
δ ← [R + γ · máx_a Q(S', a)] − Q(S, A)
Q(S, A) ← Q(S, A) + α · δ
```

Con relación al método de Q-Learning (Algoritmo 6), en esta ocasión mostramos solo la función step() dado que, al ser un algoritmo de diferencias temporales, la actualización de los valores Q se realiza en cada paso.

2.4. Evaluación y experimentos

2.4.1. Configuración experimental

Para poner a prueba nuestros algoritmos en la práctica, hemos utilizado la librería estándar de simulación **Gymnasium** (la evolución de OpenAI Gym). Esta herramienta se encarga de gestionar toda la física y las reglas del entorno (el MDP), obligando al agente a aprender basándose únicamente en el estado en el que se encuentra y las recompensas (premios o castigos) que recibe en cada paso. Para evaluar los algoritmos bajo presión, hemos utilizado el mapa clásico llamado *CliffWalking-v0*. Este entorno es una cuadrícula (*grid*) de 4 filas por 12 columnas. El objetivo del agente es ir desde la esquina inferior izquierda (coordenada [3, 0]) hasta la esquina inferior derecha (meta en [3, 11]). Dar un paso normal en cualquier dirección le cuesta -1 punto (lo que le empuja a buscar el camino más corto). Sin embargo, si da un mal paso y cae por el acantilado (la fila inferior), recibe un castigo masivo de -100 puntos y vuelve a la casilla de salida. Esta mezcla entre buscar la ruta rápida y el peligro extremo de caerse hace que este mapa sea perfecto para evaluar si un algoritmo es arriesgado o conservador.

Para asegurar que todos los experimentos sean reproducibles y comparables de forma justa, hemos fijado la semilla aleatoria (*seeding*) en las librerías `numpy` y `gymnasium` durante todas las ejecuciones. Además, salvo que se especifique lo contrario en algún experimento concreto, los parámetros de configuración (hiperparámetros) por defecto utilizados son los siguientes: un factor de descuento $\gamma = 0.95$, una tasa de aprendizaje $\alpha = 0.5$, y un límite máximo de 1000 pasos por episodio para evitar que el agente se quede atrapado en bucles infinitos. Finalmente, para obtener resultados estadísticos robustos, las medias y gráficas se han calculado simulando 10 ejecuciones de 400 episodios cada una.

2.4.2. Métricas de desempeño

El control analítico se realiza mediante una tripleta de métricas diseñadas ad-hoc en el módulo `plotting`:

1. **Recompensa Total Promediada ($\mathbb{E}[G_t]$):** Representa la media de la recompensa obtenida en cada episodio

entre todas las ejecuciones. Al igual que el resto de medidas, se puede representar utilizando una media móvil para ver la tendencia eliminando el ruido, y en muchas ocasiones se muestra con una escala logarítmica en el eje Y dado el enorme rango de las recompensas obtenidas.

2. **Longitud Temporal del Episodio ($|t|$):** Muestra el número promedio de pasos que el agente tarda en completar cada episodio, sirviendo para identificar cuándo un agente ha producido una política estacionaria sin paseos erráticos.
3. **Varianza y Error de Entrenamiento Absoluto Aislado ($\sum \mid \delta_t \mid$):** Mide la fluctuación interna inestable de una formulación temporal mal parametrizada o incapaz de estabilizar la matriz $\mathbf{Q}(s, a)$.

2.4.3. Análisis de resultados

Diferencias Temporales Off-Policy frente a On-Policy

Al someter ambos algoritmos al mapa *CliffWalking*, se hace evidente la diferencia de diseño descrita teóricamente por Sutton [1]. El algoritmo *Q-Learning* es mucho más agresivo e inestable: aprende a caminar pegado al borde del acantilado porque asume de forma optimista que siempre tomará el mejor camino posible. En la práctica, esto hace que cuando la exploración aleatoria ($\epsilon = 0.1$) le hace dar un paso al azar, caiga frecuentemente al vacío y sea gravemente penalizado. Por el contrario, *SARSA* aprende una ruta menos perfecta acercándose a la meta, pero asume esos errores durante el aprendizaje y opta por trazar un camino más largo pero mucho más seguro y alejado del precipicio.

El análisis de recompensas estables (promedio de convergencia sobre los últimos 50 episodios) lo ilustra numéricamente, exhibiendo que un enfoque *On-Policy* castiga estadísticamente menos en interacciones crudas.

- **Media Q-Learning**: ≈ -55.90 iteraciones ponderadas de castigo.
- **Media SARSA**: ≈ -26.59 iteraciones ponderadas de castigo.

Esta diferencia en las recompensas promedio se puede apreciar claramente en la Figura 2.1.

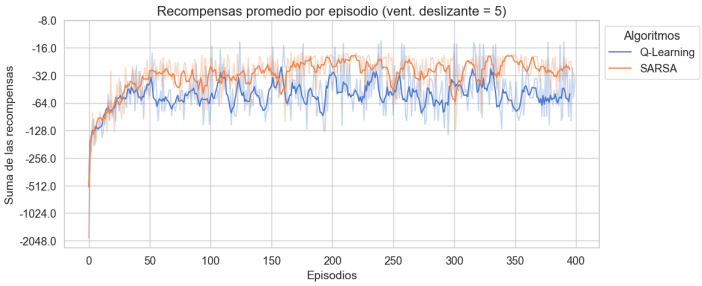
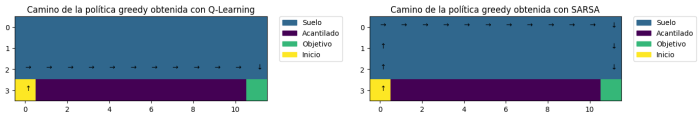


Figura 2.1: Recompensas Promedio: SARSA vs Q-Learning

Si nos fijamos en la ruta final que cada algoritmo decide tomar en el mapa (la política *greedy* aprendida tras aprender los valores de la matriz $\mathbf{Q}_*$), podemos ver claramente cómo esas diferencias de diseño que acabamos de explicar se traducen en la práctica (Figura 2.2).



(a) Ruta arriesgada Q-Learning (b) Ruta conservadora SARSA

Figura 2.2: Comparación de las rutas derivadas de la política greedy de SARSA y Q-Learning

Escalado del Ratio de Aprendizaje y Robustez Analítica

Una vez vista la superioridad de SARSA con respecto a Q-Learning en lo que a recompensas promedio se refiere, pasamos a comparar la calidad de este algoritmo con su variante SARSA esperado (*Expected SARSA*). En concreto, el experimento consistió en aumentar la tasa de aprendizaje α desde 0.1 hasta 0.9 y analizar el promedio de recompensas de cada algoritmo en el último 10 % de los episodios, para cada valor de α. En este estudio pudimos confirmar la superioridad del método *Expected SARSA*: al basarse en promedios y no depender de tener buena suerte al explorar, resulta mucho más estable, independientemente del valor de la tasa de aprendizaje. Como se aprecia en la Figura 2.3, si forzamos el cerebro del agente exigiéndole aprender demasiado rápido (subiendo $\alpha = 0.9$), el algoritmo SARSA tradicional se rompe y empieza a cometer errores catastróficos perdiendo hasta ≈ -200 puntos. Por el contrario, *Expected SARSA* resiste perfectamente este estrés, amortiguando los saltos y manteniéndose estable en apenas ≈ -15 puntos.

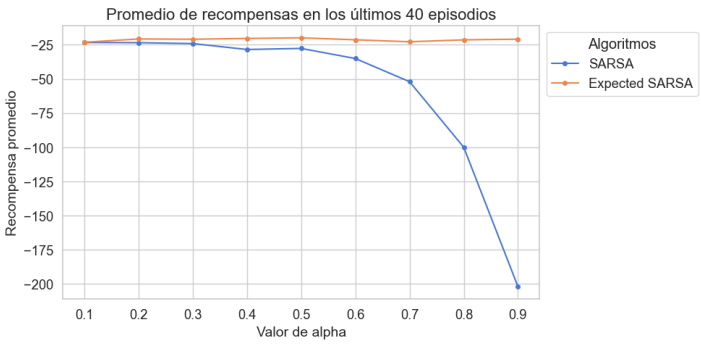


Figura 2.3: Sensibilidad paramétrica α en variantes SARSA

La Figura 2.4 muestra la ruta generada empleando la política *greedy* generada a partir de la tabla $\mathbf{Q}$ obtenida mediante *Expected SARSA*. Resulta francamente curioso cómo este algoritmo se posiciona en un punto intermedio entre Q-Learning y SARSA, produciendo una ruta que no es ni la más arriesgada ni la más cautelosa.

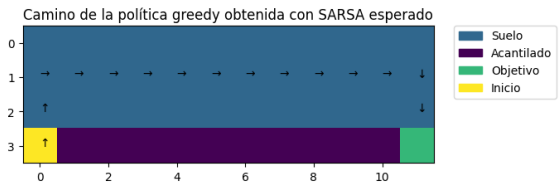


Figura 2.4: Ruta derivada de la política greedy de SARSA esperado

Rendimiento de n-step SARSA en función del número de pasos

Un compromiso fundamental que se produce dentro del aprendizaje por refuerzo es el *trade-off* entre sesgo y varianza. Aquellos métodos que consideran más pasos temporales para realizar una actualización de la tabla $\mathbf{Q}$, como Monte Carlo, presentan una varianza superior a los métodos que actualizan su matriz en cada paso de ejecución, pero a cambio tienen un menor sesgo. Esto significa que, aunque no sobreestiman ni subestiman la calidad de un par (S, A), su tiempo de convergencia es mayor y resultan más inestables, especialmente en los primeros episodios. Para mostrar cómo afecta el número de pasos temporales al rendimiento del algoritmo, se ejecutó el método de SARSA de n pasos variando el parámetro n para distintos valores de α, pudiendo medir así la robustez de cada variante a la tasa de aprendizaje. El resultado viene reflejado en la Figura 2.5.

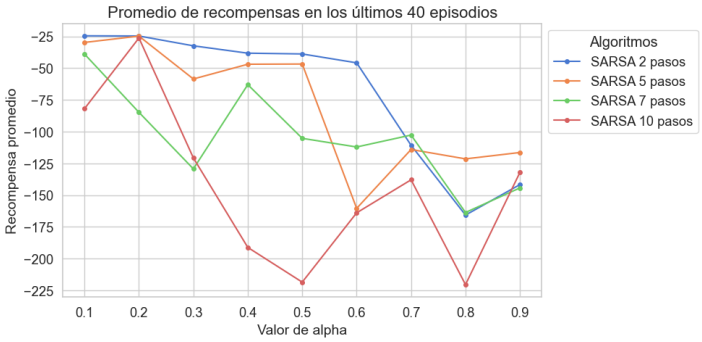


Figura 2.5: Rendimiento de SARSA de n pasos en función de n y de la tasa de aprendizaje

Tal y como podemos ver en la anterior figura, SARSA de 2 pasos sobresale frente al resto de variantes para prácticamente todos los valores de α. De hecho, se aprecia una tendencia clara: a menor número de pasos, mayor recompensa promedio. Solo en los valores más elevados de la tasa de aprendizaje se puede observar un sutil cambio, dado que SARSA de 5 pasos comienza a ser ligeramente mejor que SARSA de 2 pasos, y SARSA de 10 pasos mejora sus resultados con respecto a SARSA de 7 pasos. También se aprecia que un mayor valor de n se traduce en un aumento de la varianza, puesto que la curva de recompensas para $n = 10$ parece ser la más inestable de todas, mientras que un valor de $n = 2$ tiene una tendencia menos errática.

Solución al problema del optimismo excesivo (Maximization Bias)

Al evaluar el algoritmo *Double Q-Learning* propuesto por van Hasselt [5], observamos que el agente desdobla su cerebro en dos memorias separadas $(\mathbf{Q}_1, \mathbf{Q}_2)$. Esto le permite evitar el principal defecto del Q-Learning normal: la tendencia a sobrevalorar rutas que parecen buenas al principio pero que en realidad son trampas mortales. Sin embargo, en la práctica, forzar al agente a mantener dos memorias separadas hace que su aprendizaje inicial sea excesivamente lento. Si mantenemos una exploración fija (como $\epsilon = 0.1$), el agente necesita jugar más de 8000 pasos para empezar a aclararse y converger. Para solucionar este enorme retraso, tuvimos que aplicarle una estrategia dinámica: programar al agente para que al principio explore muchísimo y vaya reduciendo esa exploración poco a poco (un ϵ_{decay} durante los primeros 23000 pasos de simulación hasta llegar al ϵ_{min}). Con este truco logramos mejorar significativamente la convergencia del algoritmo, pasando de 8000 episodios a apenas 2000 para lograr resultados comparables a Q-Learning (Figura 2.6). Sin embargo, sigue estando lejos de los menos de 400 episodios que Q-Learning requería para converger.

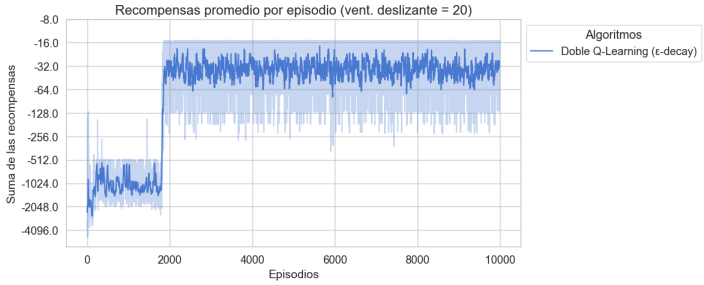


Figura 2.6: Estabilización de Double Q-Learning usando exploraciones tipo Decay

La ineficacia práctica de Monte Carlo en entornos de peligro

Nuestros experimentos demuestran el enorme retraso de aprendizaje que sufren los algoritmos que esperan al final de la partida para aprender (Monte Carlo) frente a los que aprenden paso a paso (Diferencias Temporales). Durante sus primeras 400 partidas exploratorias, *Monte Carlo On-Policy* arrastra un error gravísimo: tarda alrededor de 900 pasos intermedios en acabar cada partida porque es incapaz de sacar conclusiones sobre los castigos que recibe hasta que no alcanza un estado final y reinicia, a diferencia de algoritmos como SARSA esperado, que logran alcanzar rápidamente un número de pasos cercano al óptimo (Figura 2.7). Por tanto, su uso resulta totalmente ineficaz en mapas de tipo *GridWorld* donde abundan las penalizaciones severas, demostrando que aprender paso a paso es una estrategia muy superior.

Figura 2.7: Lentitud de la convergencia de métodos de Monte Carlo frente a Diferencias Temporales

Estancamiento de MC On Policy frente a MC Off Policy

A pesar de la superioridad notable de los métodos de Diferencias Temporales, realizamos un último experimento para sacar a relucir las diferencias entre diversas versiones del método de Monte Carlo, en particular, comparamos MC On Policy de primera visita, de todas las visitas, y MC Off Policy con muestreo de importancia ponderado. A la luz de los resultados, realizamos las siguientes observaciones: las dos versiones

de MC On Policy presentan una evolución de recompensas prácticamente idéntica durante los primeros 230 episodios, momento a partir del cual MC de primera visita comienza a superar a MC de todas las visitas no solo en cuanto a recompensas, sino también en cuanto a longitud promedio de los episodios. Además, el error de entrenamiento de MC de primera visita se mantiene permanentemente por debajo del error de MC de todas las visitas, lo cual probablemente se debe al hecho de que el primer método usa estimadores insesgados, mientras que Monte Carlo de todas las visitas tiene un sesgo mayor.

En cuanto a Monte Carlo Off Policy, apreciamos que presenta una mayor varianza, lo cual encaja perfectamente con la teoría, que nos dice que las estimaciones del muestreo de importancia pueden tener varianza infinita (no acotada), si bien el muestreo ponderado la reduce significativamente. Además, se observa que MC Off Policy comienza en recompensas bastante inferiores a los otros dos métodos, pero su velocidad de aprendizaje es mucho mayor, alcanzando a MC On Policy en los últimos episodios tal y como se ve en la Figura 2.8.

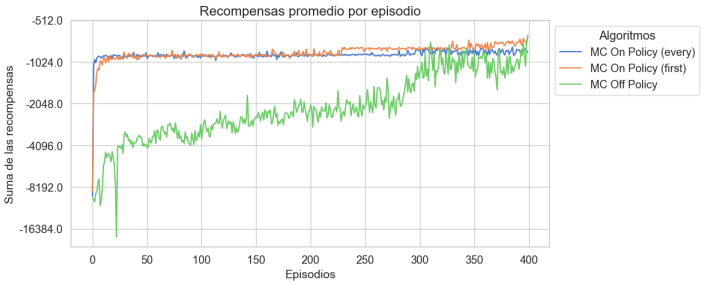


Figura 2.8: Comparativa de tres versiones diferentes del método de Monte Carlo

2.5. Conclusiones

2.5.1. Resumen de los principales resultados obtenidos

A lo largo de este proyecto, hemos comprobado que los métodos tabulares son muy efectivos para resolver problemas de control discreto estocástico en entornos con estados y acciones finitas. Las pruebas realizadas en el entorno *CliffWalking* han demostrado la diferencia de comportamientos dentro de la familia de algoritmos que aprenden paso a paso (Diferencias Temporales). Por un lado, *Q-Learning* es el más rápido descubriendo el trayecto más corto, pero en la práctica acaba sufriendo graves penalizaciones porque, debido a su excesivo optimismo matemático (*Maximization Bias*) y a los pasos estocásticos dados al azar en la exploración, cae demasiadas veces por el acantilado.

En contraste, *Expected SARSA* ha demostrado ser el algoritmo más seguro y equilibrado. Al calcular los siguientes pasos basándose en promedios y probabilidades $\mathbb{E}[Q(s', a)]$, consiguió eliminar las peligrosas oscilaciones que sufría el *SARSA* tradicional cuando le forzábamos a aprender demasiado deprisa (aumentando su tasa de aprendizaje α). Por su parte, los algoritmos clásicos como *Monte Carlo*, que necesitan esperar al final de la partida para sacar conclusiones, demostraron ser demasiado lentos y en la práctica totalmente inútiles para mapas tan castigadores y peligrosos.

2.5.2. Limitaciones del estudio

A pesar de que estos algoritmos funcionan bien cuando el problema es pequeño, tienen un límite técnico insalvable. El problema fundamental es de memoria de ordenador: los algoritmos tabulares obligan a crear una enorme tabla o matriz interactiva ($\mathbf{Q} \in \mathbb{R}^{|\mathcal{S}| \times |\mathcal{A}|}$) donde deben guardar de antemano todas las combinaciones posibles de estado y acción. Si usáramos estos algoritmos en un videojuego moderno donde las variables son infinitas, la memoria RAM colapsaría al instante. Esto es conocido en la Inteligencia Artificial como la "Maldición de la Dimensionalidad" (*The Curse of Dimensionality*). Además, en este proyecto hemos asumido que las reglas del tablero son siempre fijas y estáticas $\langle \mathcal{S}, \mathcal{A}, \mathcal{P}, \mathcal{R}, \gamma \rangle$, dejando de lado los cambios imprevistos o trampas dinámicas que suelen aparecer en los problemas reales del mundo de la industria.

2.5.3. Reflexión sobre la importancia y el impacto en RL

Haber construido una arquitectura modular pura, donde el algoritmo o `Learner` está totalmente separado del simulador *Gymnasium*, tiene un enorme valor para comprender el funcionamiento interno de los algoritmos de aprendizaje por refuerzo. Estos algoritmos clásicos son uno de los pilares fundamentales sobre los que se sostiene la Inteligencia Artificial moderna. Entender las diferencias entre aprender sobre la marcha (*On-Policy*) o de forma disociada (*Off-Policy*), y saber cómo arreglar el exceso de optimismo con *Double Q-Learning*, es un paso obligatorio antes de enfrentarnos a sistemas modernos más complejos como las Redes Neuronales Profundas (*Deep Q-Networks* o DQN). De hecho, los mecanismos avanzados que usan esas redes, como las *Target Networks*, no son más que la evolución directa de los conceptos de predicción paso a paso (*bootstrapping*) que hemos puesto a prueba en este capítulo.

2.5.4. Líneas futuras de estudio

Vistos los límites evidentes de intentar memorizar cada casilla en una tabla $\mathbf{Q}(S, A)$, el siguiente paso natural para futuras investigaciones será utilizar métodos que permitan al agente "generalizar" su conocimiento. El próximo gran salto en desarrollo será abandonar esa inmensa tabla rígida y sustituirla por fórmulas matemáticas o ecuaciones (un Vector de Pesos $\mathbf{w}$) mediante la **Aproximación de Funciones**. Incorporar herramientas como el descenso de gradiente (*Semi-Gradient SARSA*) o incluso redes neuronales sencillas, permitirá solucionar definitivamente el problema de colapso de memoria RAM. Esto otorgará al agente una capacidad fundamental: el poder de generalizar. Es decir, podrá deducir cuál es la mejor acción a tomar en situaciones o partes del mapa que nunca ha llegado a visitar, simplemente aplicando la lógica que ha aprendido en escenarios similares.

Capítulo 3

Entornos Complejos: Métodos Aproximados

3.1. Introducción

3.1.1. Descripción del problema y relevancia

Como ya se ha explicado en capítulos anteriores, el Aprendizaje por Refuerzo (RL, del inglés *Reinforcement Learning*) estudia cómo un agente autónomo aprende a tomar decisiones óptimas interactuando con un entorno mediante el esquema de prueba y error. En su formulación estándar como **Proceso de Decisión de Markov** (MDP), el agente observa el estado del entorno $s_t \in \mathcal{S}$, selecciona una acción $a_t \in \mathcal{A}$ y recibe una recompensa escalar $r_{t+1} \in \mathbb{R}$, con el objetivo de maximizar el retorno acumulado descontado:

$$G_t = \sum_{k=0}^{\infty} \gamma^k R_{t+k+1}, \qquad \gamma \in [0, 1)$$

Cuando el espacio de estados $\mathcal{S}$ es **continuo** o de alta dimensionalidad, la representación tabular de la función de valor-acción $Q : \mathcal{S} \times \mathcal{A} \to \mathbb{R}$ resulta computacionalmente inviable. El problema central de los *entornos complejos* reside en cómo generalizar el conocimiento adquirido en estados visitados hacia estados no visitados de la misma región del espacio.

La relevancia de este problema es doble: por un lado, la mayoría de las aplicaciones industriales y científicas de RL involucran espacios de estados continuos (robótica, vehículos autónomos, gestión de energía, biomedicina); por otro, los avances en **aproximación de funciones** —especialmente mediante redes neuronales profundas— han impulsado logros como el control a nivel humano en videojuegos Atari [6] o el dominio de juegos de mesa complejos [7].

3.1.2. Motivación del trabajo

El punto de partida de este capítulo es el límite de escalabilidad evidenciado en el Capítulo 2 (métodos tabulares). Por ejemplo, dado un entorno como `MountainCar-v0` con un espacio de estados $\mathcal{S} \subseteq \mathbb{R}^2$, la discretización en una rejilla de $20 \times 20 = 400$ celdas muestra que el agente requiere un presupuesto de episodios muy elevado para mejorar de forma apreciable, ya que $\hat{Q}(s, a)$ solo actualiza la celda $\lfloor s/\Delta \rfloor$, pero no generaliza a otras celdas de estados "adyacentes".

Esta rigidez estructural motiva el estudio de métodos aproximados, que sustituyen la tabla por una función paramétrica $\hat{q}(s, a\,; \boldsymbol{\theta})$ capaz de interpolar entre estados no visitados. La motivación específica de este trabajo es:

1. **Superar la barrera de escalabilidad** de los métodos tabulares en entornos con observaciones continuas.
2. **Comparar el espectro de aproximadores**: desde la aproximación lineal con Tile Coding hasta las redes neuronales profundas (DQN, Double DQN).
3. **Caracterizar el compromiso** entre capacidad expresiva del aproximador, estabilidad del entrenamiento y eficiencia muestral.
4. **Evaluar el papel de la infraestructura Gymnasium** como marco estándar de experimentación reproducible en RL.

3.1.3. Objetivos del informe

Este capítulo persigue los siguientes objetivos:

1. Formalizar el marco matemático de la aproximación de funciones en RL.
2. Describir e implementar tres familias de métodos aproximados: SARSA Semi-Gradiente con Tile Coding, Deep Q-Learning (DQN) y Double DQN.
3. Diseñar y ejecutar un protocolo experimental reproducible sobre el entorno `CartPole-v1`.
4. Analizar cuantitativamente los resultados y comparar las familias algorítmicas.
5. Extraer conclusiones sobre la adecuación de cada aproximador.

3.1.4. Organización del capítulo

El resto del capítulo se estructura como sigue: la Sección 3.2 presenta la definición formal del problema y los trabajos relacionados; en el apartado 3.3 se detallan los pseudocódigos y ecuaciones de los tres algoritmos; la Sección 3.4 describe la configuración experimental, métricas y resultados; y finalmente la Sección 3.5 sintetiza los hallazgos, limitaciones y líneas futuras.

3.2. Desarrollo

3.2.1. Definición formal del problema

El problema de control en entornos complejos se formula como un Proceso de Decisión de Markov (MDP):

$$\mathcal{M} = \langle \mathcal{S},\, \mathcal{A},\, \mathcal{P},\, \mathcal{R},\, \gamma \rangle$$

La función de valor-acción óptima Q^* satisface las Ecuaciones de Bellman de Optimalidad:

$$\begin{aligned} Q^*(s,a) &= \mathcal{R}(s,a) + \gamma \sum_{s' \in \mathcal{S}} \mathcal{P}(s' \mid s,a) \max_{a'} Q^*(s',a') \\ &= (\mathcal{T}^* Q^*)(s,a) \end{aligned}$$

El operador $\mathcal{T}^*$ es una contracción en norma ℓ_∞ con factor $\gamma < 1$, garantizando la existencia y unicidad del punto fijo [8]. Cuando $|\mathcal{S}|$ es inmanejable, se busca $\hat{q}(\cdot\,;\boldsymbol{\theta}) \in \mathcal{F}_{\boldsymbol{\theta}}$ tal que:

$$\hat{q}(\cdot\,;\boldsymbol{\theta}) \approx Q^*, \qquad \boldsymbol{\theta} \in \mathbb{R}^p,\ p \ll |\mathcal{S}| \cdot |\mathcal{A}|$$

3.2.2. Trabajos relacionados

La aproximación de funciones en RL tiene sus raíces en los trabajos fundacionales de Sutton (1988) sobre el aprendizaje por Diferencias Temporales (TD) y la tesis de Watkins (1989) sobre Q-Learning.

Aproximación lineal y Codificación de características

Sutton y Barto [1] formalizan el **SARSA Semi-Gradiente** con aproximación lineal, demostrando que se produce una convergencia al óptimo bajo la hipótesis de que la función de distribución de visitas a estados $\mu(s)$ es estacionaria. El uso de Tile Coding como esquema de representación permite aprovechar la dispersión (*sparsity*) del vector de características: solo m de los d componentes son activos en cada paso, reduciendo la evaluación de $O(d)$ a $O(m)$. De esta forma, en cada paso no se tiene que actualizar el vector entero, pero tampoco se actualizan las "celdas" de una en una, acelerando así la convergencia.

Deep Q-Network (DQN)

Mnih et al. [6] introdujeron DQN, que combina Q-Learning con redes neuronales convolucionales para aprender directamente desde píxeles en 49 juegos Atari. Sus contribuciones técnicas clave fueron:

- **Experience Replay**: almacenamiento de transiciones en un buffer $\mathcal{D}$ y muestreo aleatorio de mini-batches para romper correlaciones temporales.
- **Red Target** (*target network*): copia periódica de los pesos $\boldsymbol{\theta}^- \leftarrow \boldsymbol{\theta}$ para estabilizar los targets de Bellman durante el entrenamiento.

Double DQN

Van Hasselt et al. [9] identificaron que DQN estándar sobreestima sistemáticamente los valores Q debido al operador máx en el cálculo del target:

$$\mathbb{E}\Big[\max_{a'} Q_{\boldsymbol{\theta}}(s',a')\Big] \geq \max_{a'} \mathbb{E}[Q_{\boldsymbol{\theta}}(s',a')]$$

Su solución, **Double DQN**, desacopla la *selección* de acción (red online θ) de la *evaluación* de su valor (red target θ^-), de manera similar a como sucede en el método de Doble Q-Learning estándar.

3.2.3. Explicación del enfoque y los métodos utilizados

El proyecto implementa el patrón Learner–Policy–Agent con separación estricta de responsabilidades. La clase `Agent` orquesta el bucle de entrenamiento de forma agnóstica al algoritmo. Para SARSA Semi-Gradiente, el módulo `TileCodingEnv` actúa como un *Wrapper* de Gymnasium, transformando la observación continua $\mathbf{s} \in \mathbb{R}^4$ en el vector de características binario $\boldsymbol{\phi}(\mathbf{s}) \in \{0,1\}^d$.

Se implementan tres familias de métodos aproximados en orden creciente de complejidad:

1. **Aproximación Lineal con Tile Coding (SARSA Semi-Gradiente)**: $\hat{q}(\mathbf{s},a\,;\mathbf{W}) = \mathbf{w}_a^\top \boldsymbol{\phi}(\mathbf{s})$, con $d = 4 \cdot 8^4 = 16\,384$. Convergencia demostrada bajo condiciones de regularidad. Limitación: no puede representar interacciones no lineales.
2. **Deep Q-Network (DQN)**: $Q_{\boldsymbol{\theta}} : \mathbb{R}^4 \to \mathbb{R}^2$ con arquitectura $4 \to 64 \to 2$ (ReLU). Capacidad no lineal con Experience Replay para estabilidad del gradiente.
3. **Double DQN**: Extensión de DQN con desacoplamiento Selección–Evaluación para reducir el sesgo de sobreestimación positivo del operador máx.

3.2.4. Diferenciación respecto a métodos tabulares

La Tabla 3.1 recoge un resumen de las diferencias fundamentales entre los métodos aproximados analizados en el Capítulo 2 y los métodos aproximados que se estudian en el presente capítulo.

3.3. Algoritmos

Esta sección detalla los tres algoritmos implementados en el proyecto, describiendo sus pasos, ecuaciones de actualización, pseudocódigos formales y la justificación de su elección.

3.3.1. Construcción del vector de características: Tile Coding

En los métodos lineales como SARSA semi-gradiente es fundamental elegir una codificación adecuada para representar los estados de manera discreta mediante un vector de características. Resulta especialmente interesante utilizar representaciones que permitan capturar interacciones entre varias características, de tal forma que la actualización de un estado suponga la actualización de no uno, sino varios elementos del vector. Con tal propósito, se han ideado multitud de codificaciones diferentes, como bases de Fourier o codificación gruesa, pero en este proyecto se ha implementado la codificación por **teselados** (*tile coding*).

El funcionamiento de la codificación por teselados es el siguiente: se realiza una partición del espacio de observaciones en teselas y posteriormente se desplaza este teselado en una fracción del tamaño de la tesela, generando así varios teselados superpuestos (pero con un *offset*). El vector de características $\mathbf{x}(s)$ tendrá una componente por cada tesela de cada teselado, y el número de características activas para cada estado será

Tabla 3.1: Comparación entre métodos tabulares (Cap. 2) y métodos aproximados (Cap. 3)

Dimensión	**Métodos Tabulares**	**Presente Trabajo**
Espacio de estados	Discreto, finito	Continuo, $\mathcal{S} \subseteq \mathbb{R}^4$
Representación de Q	Tabla $\mathbf{Q} \in \mathbb{R}^{\lvert\mathcal{S}\rvert \times \lvert\mathcal{A}\rvert}$	Función paramétrica $\hat{q}(\cdot; \boldsymbol{\theta})$
Generalización	Ninguna (estado a estado)	Implícita (vecindad en $\mathcal{S}$)
Número de parámetros	$\lvert\mathcal{S}\rvert \cdot \lvert\mathcal{A}\rvert$	$p \ll \lvert\mathcal{S}\rvert \cdot \lvert\mathcal{A}\rvert$
Entorno de evaluación	`FrozenLake-v1` (16 estados)	`CartPole-v1` ($\mathcal{S} \subseteq \mathbb{R}^4$)

exactamente igual al número de teselados (m). Es decir, a la hora de representar un estado, se debe determinar el índice de la tesela (o *bin*) en la que "cae" el estado para cada uno de los m teselados.

Por último, hemos de tener en cuenta que, dado que el problema que queremos resolver es de control y no de predicción, en el vector de características también depende de la acción tomada en cada paso, de modo que nos referiremos a este vector como $\mathbf{x}(s, a)$. El número total de parámetros que hemos descrito previamente ha de ser multiplicado, por tanto, por la cantidad de acciones (discretas) que se pueden tomar.

Los parámetros de Tile Coding que hemos utilizado en nuestros experimentos se detallan en la Tabla 3.2.

Tabla 3.2: Parámetros de Tile Coding en el experimento

Parámetro	**Símbolo**	**Valor**
Número de teselados	m	4
Bins (teselas) por dimensión	n_j	8, $\forall j$
Teselas por teselado	N_{tile}	$8^4 = 4\,096$
Dimensión del vector	$d = m \cdot N_{\text{tile}}$	16 384
Tasa normalizada	$\alpha_{\text{eff}} = \alpha/m$	0.025

3.3.2. SARSA Semi-Gradiente con Tile Coding

SARSA Semi-Gradiente es un método On Policy de Diferencias Temporales de un paso con aproximación lineal. La regla de actualización es:

$$\mathbf{w} \leftarrow \mathbf{w} + \alpha_{\text{eff}} \cdot \delta_t \ \cdot \nabla \hat{q}(S, A, \mathbf{w})$$

con el error TD On Policy:

$$\delta_t = [R + \gamma\, \hat{q}(S', A', \mathbf{w})] - \hat{q}(S, A, \mathbf{w})$$

La aproximación de los valores de cada par (S, A) se calcula de la siguiente manera:

$$\hat{q}(S, A, \mathbf{w}) = \mathbf{w}^T \cdot \mathrm{x}(S, A)$$

El proceso de este algoritmo queda recogido en Algorithm 7.

Algorithm 7 SARSA Semi-Gradiente

Require: env, m=4, α=0.1, γ=0.95, ε=0.1, $E = 400$

```
w ← 0;  α_eff ← α/m
for episodio e = 1, ..., E do
    S ← env.reset()                    ▷ Vía TileCodingEnv
    A ← ε-greedy(w^T · s)
    while not done do
        S', R ← env.step(A)
        A' ← ε-greedy(w^T · s')
        δ ← [R + γ q̂(S', A', w)] − q̂(S, A, w)
        w ← w + α_eff · δ_t · ∇q̂(S, A, w)
        S ← S';  A ← A'
    end while
end for
return w
```

3.3.3. Deep Q-Network (DQN)

DQN [6] extiende Q-Learning a aproximadores no lineales mediante *Experience Replay* y una red *target*. En nuestro caso, la arquitectura es $Q_{\boldsymbol{\theta}} : \mathbb{R}^4 \to \mathbb{R}^2$ dado que la capa de entrada debe tener tantos nodos como variables observables tenga el entorno (en nuestro caso, 4), y la capa de salida contiene tantos nodos como acciones pueda tomar el agente (dos en nuestro caso). Además, empleamos dos capas ocultas con 64 nodos cada una. Por tanto, el cálculo de los valores Q para cada acción, dado un estado S, se realiza de la siguiente forma:

$$\begin{aligned}
\mathrm{h}^{(1)} &= \mathrm{ReLU}\Big(\mathrm{W}^{(1)} S + \mathrm{b}^{(1)}\Big), \quad \mathrm{W}^{(1)} \in \mathbb{R}^{64\times 4} \\
\mathrm{h}^{(2)} &= \mathrm{ReLU}\Big(\mathrm{W}^{(2)} \mathrm{h}^{(1)} + \mathrm{b}^{(2)}\Big), \quad \mathrm{W}^{(2)} \in \mathbb{R}^{64\times 64} \\
\mathbf{Q}_{\theta}(S) &= \mathrm{ReLU}\Big(\mathrm{W}^{(3)} \mathrm{h}^{(2)} + \mathrm{b}^{(3)}\Big), \quad \mathrm{W}^{(3)} \in \mathbb{R}^{2\times 64}
\end{aligned}$$

La función de pérdida MSE sobre el mini-batch $\mathcal{B} \overset{\text{i.i.d.}}{\sim} \mathcal{D}$:

$$\begin{aligned}
\mathcal{L}(\boldsymbol{\theta}; \mathcal{B}) &= \frac{1}{\lvert\mathcal{B}\rvert} \sum_{i=1}^{\lvert\mathcal{B}\rvert} \left(y_i - Q_{\boldsymbol{\theta}}(s_i)\right)^2, \\
y_i &= r_i + \gamma \max Q_{\boldsymbol{\theta}}(s_i')
\end{aligned}$$

Los pesos se actualizan con el optimizador Adam con los parámetros $\eta = 10^{-3}$, $\beta_1 = 0.9$, $\beta_2 = 0.999$. El algoritmo se resume en el pseudocódigo de Algorithm 8.

3.3.4. Double Deep Q-Network (DDQN)

Double DQN [9] corrige el sesgo de sobreestimación desacoplando la selección y evaluación de la acción óptima mediante la **red target** $\boldsymbol{\theta}^-$:

$$y_i^{\text{DDQN}} = r_i + \gamma\,(1 - d_i)\, Q_{\boldsymbol{\theta}^-}\left(s_i',\ \underbrace{\arg\max_{a'} Q_{\boldsymbol{\theta}}(s_i', a')}_{\text{selección: red online } \boldsymbol{\theta}}\right)$$

La red target se actualiza con copia *hard* cada $\tau = 100$ pasos: $\boldsymbol{\theta}^- \leftarrow \boldsymbol{\theta}$. Este método se resume en Algorithm 9.

Tabla 3.3: Criterios de selección de los algoritmos

Criterio	**SARSA SG**	**DQN**	**DDQN**
Espacio continuo $\mathcal{S} \subseteq \mathbb{R}^4$	✓	✓	✓
Convergencia demostrada	✓	Parcial	Parcial
Corrección de sobreestimación	—	—	✓
Generalización no lineal	—	✓	✓
Eficiencia muestral	Alta ($O(m)$)	Media	Media

Algorithm 8 Deep Q-Network (DQN)

Require: env, tasa de aprendizaje η=10^{-3}, tasa de descuento γ=0.95, ε=0.1, tamaño de lote B=64, tamaño de *Replay Memory* $|\mathcal{D}|_{\text{máx}}$=10000, $E = 400$
Ensure: $\boldsymbol{\theta}$ (pesos de la Q-red)

```
θ ← Xavier-init
D ← ∅
for episodio e = 1, ..., E do
    S ← env.reset()
    while not done do
        A ← ε-greedy(Q_θ(S))
        S', R ← env.step(A)
        D ← D ∪ {(S, A, R, S')}          ▷ Replay Memory
        if |D| ≥ B then
            B ← SampleUniform(D, B)
            y_i ← r_i + γ máx Q_θ(s'_i) for (s_i, a_i, r_i, s'_i) ∈ B
            θ ← Adam(θ, ∇_θ L)
        end if
        S ← S'
    end while
end for
return θ
```

Algorithm 9 Double Deep Q-Network (DDQN)

Require: env, η=10^{-3}, γ=0.95, ε=0.1, B=64, τ=100, E = 400
Ensure: $\boldsymbol{\theta}$ (pesos de la red online)

```
θ ← Xavier-init
θ⁻ ← θ;  D ← ∅;  t_g ← 0
for episodio e = 1, ..., E do
    s ← env.reset()
    while not done do
        a ← ε-greedy(Q_θ(s))
        S', R ← env.step(A)
        D ← D ∪ {(S, A, R, S')}
        if |D| ≥ B then
            B ← SampleUniform(D, B)
            for all (s_i, a_i, r_i, s'_i) ∈ B do
                a*_i ← arg máx Q_θ(s'_i)
                y_i ← r_i + γ Q_θ⁻(s'_i)_{a*_i}
            end for
            θ ← Adam(θ, ∇_θ L)
        end if
        t_g ← t_g + 1
        if (t_g mód τ) == 0 then θ⁻ ← θ
        end if
        S ← S'
    end while
end for
return θ
```

3.3.5. Justificación de los algoritmos elegidos

A la hora de evaluar los algoritmos, SARSA Semi-Gradiente actúa como **línea base** clásica. Deep Q-Learning es el representante canónico del RL profundo basado en valor, y debería mejorar el rendimiento de SARSA SG. Finalmente, Double DQN aísla empíricamente el efecto del sesgo de sobreestimación, al compartir arquitectura y todos los hiperparámetros con DQN. Los criterios de selección se resumen en la Tabla 3.3.

3.4. Evaluación y experimentos

3.4.1. Configuración experimental

Entorno de Prueba

El entorno `CartPole-v1` de Gymnasium implementa el problema clásico de control de un péndulo invertido. El estado es el vector $\mathbf{s} = (x,\, \dot{x},\, \theta,\, \dot{\theta})^\top \in \mathbb{R}^4$, donde las cuatro variables observables se definen de la siguiente manera:

- Posición del carro: $x \in [-4.8, 4.8]$
- Velocidad lineal del carro: $\dot{x} \in [-\infty, +\infty]$
- Ángulo del palo en radianes: $\theta \in [-0.418, 0.418]$
- Velocidad angular del palo: $\dot{\theta} \in [-\infty, +\infty]$

En cada paso temporal, el agente puede tomar dos acciones: empujar el carro hacia la derecha o hacia la izquierda. Por cada paso que el agente logre mantener el palo sobre el carro sin que se caiga o se incline demasiado, éste recibe una recompensa de $+1$, de modo que su objetivo será mantener el palo sobre el carro durante el mayor número posible de pasos para maximizar así su recompensa.

Un episodio finaliza cuando se produce alguna de las siguientes condiciones:

1. El ángulo de la pértiga es mayor que ± 12.
2. La posición del carro es mayor que ± 2.4 (es decir, el centro del carro llega al límite de la imagen).
3. El número de episodio es mayor que el límite establecido (en nuestro caso, 1000).

CartPole es uno de los *benchmarks* estándar para validar métodos aproximados de RL: su estado continuo hace inviable la tabulación, pero su dinámica es suficientemente simple para converger en unos pocos cientos de episodios.

Protocolo Experimental

En la Tabla 3.4 indicamos los parámetros generales que hemos utilizado para todos los experimentos. La reproducibilidad se

garantiza mediante la función `set_seed`(σ). Los parámetros de la codificación con teselados se detallaron en la Tabla 3.2. En cuanto a los métodos de la familia Deep Q-Learning, utilizamos una red neuronal de 2 capas ocultas con 64 neuronas cada una, una *replay memory* de tamaño 10000, tasa de aprendizaje $\alpha = 10^{-3}$ y tamaño de lote de 64. En el caso particular de Doble DQN, las dos redes se sincronizan cada 100 pasos.

Tabla 3.4: Protocolo experimental

Parámetro	**Valor**
Número de ejecuciones (N_{runs})	10
Episodios por ejecución (E)	400
Semilla base (σ)	123
Factor de descuento (γ)	0.95
Política de exploración	ε-greedy, $\varepsilon = 0.1$

3.4.2. Métricas de Evaluación

La métrica principal es la **recompensa promedio** a lo largo de los 400 episodios. Dado que en cada paso temporal el agente recibe una recompensa de $+1$, el análisis de las recompensas nos da además la misma información que el análisis de longitudes promedio. Dicho de otro modo, en este entorno en particular, la longitud de un episodio equivale a la recompensa obtenida. Dado que hemos establecido que el límite de pasos de cada episodio sea 1000, este valor será la cota superior de las recompensas promedio. Si un agente obtuviese consistentemente recompensas de 1000, podríamos decir que ha logrado aprender la política óptima.

Por otra parte, también analizamos la pérdida de entrenamiento promedio por episodio, que en el caso de DQN y Double DQN se calcula como:

$$\bar{\mathcal{L}}_e = \frac{1}{|E|} \sum_{t \in |E|} \mathcal{L}(\boldsymbol{\theta}_t; \mathcal{B}_t) \tag{3.1}$$

3.4.3. Resultados Obtenidos

Recompensas promedio

En la gráfica de recompensas promedio de SARSA Semi-Gradiente (curva azul en la Figura 3.1) se aprecia que durante los 150 primeros episodios el algoritmo se encuentra en un estado de estancamiento generalizado, sin superar los 16 puntos de recompensa (un valor paupérrimo con respecto al valor óptimo de 1000). No obstante, a partir de dicho momento, el agente comienza a aprender a un ritmo bastante rápido, alcanzando recompensas de más de 70 puntos hacia el final del proceso. De hecho, dada la inclinada pendiente de la curva en los últimos episodios, da la impresión de que el algoritmo aun no ha convergido y se beneficiaría de una ampliación del número de episodios de aprendizaje.

Con respecto a los métodos de la familia DQN (curvas naranja y verde en la Figura 3.1), vemos que el algoritmo de Deep Q-Learning logra una convergencia más temprana que su variante Doble DQN, al igual que nos ocurría en los métodos tabulares. A pesar de que en algunos instantes Doble DQN supera a Deep Q-Learning, podemos decir que el comportamiento de ambos métodos es muy similar, y no se aprecia una mayor estabilidad de Doble DQN como cabría esperar.

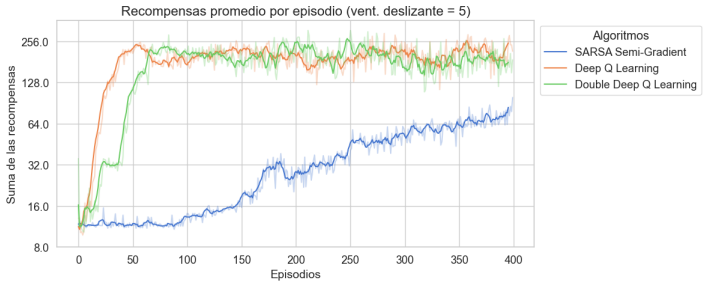


Figura 3.1: Promedio de recompensas de los métodos aproximados

Ningún algoritmo alcanza la recompensa máxima de 1000 en el horizonte de 400 episodios, así que ninguno ha logrado aprender la política óptima. Los valores medios de recompensas en los últimos 50 episodios fueron los siguientes:

- SARSA Semi-Gradiente: 69.4
- Deep Q-Learning: 218.3
- Doble DQN: 189.1

La Figura 3.1 muestra una clara superioridad de los métodos DQN en lo que respecta a velocidad de aprendizaje, sin embargo, no es posible afirmar que estos métodos sean holísticamente superiores a SARSA Semi-Gradiente. Como se aprecia en la gráfica, los algoritmos DQN parecen haberse estabilizado en recompensas promedio de alrededor de 256 puntos, mientras que SARSA Semi-Gradiente sigue aprendiendo a un ritmo veloz en los últimos episodios. Por tanto, es posible que, dado un mayor número de episodios, SARSA lograra obtener recompensas mayores que DQN.

Para verificar esta hipótesis, se realizó un entrenamiento de este algoritmo durante 800 episodios, cuyos resultados se muestran en la Figura 3.2. Como podemos ver, tras 800 episodios el algoritmo sí parece haber logrado cierto nivel de convergencia alrededor de los 140 puntos de recompensa promedio, por lo que podemos decir con mayor seguridad que los métodos DQN parecen ser superiores a SARSA no solo en cuanto a velocidad de aprendizaje, sino en cuanto a la calidad de la solución aprendida.

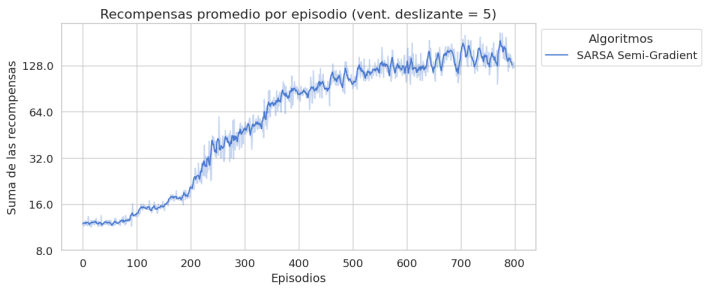


Figura 3.2: Promedio de recompensas SARSA semi-gradiente con 800 episodios

Error de entrenamiento promedio

En la Figura 3.3 podemos ver que SARSA semi-gradiente comienza en errores de entrenamiento menores que los métodos DQN pero, conforme avanzan los episodios, sus estimaciones se vuelven cada vez peores, provocando errores más grandes cuanto más largos son los episodios. Sin embargo, a pesar de que DQN parte de un pico de error que supera los 128 puntos, sus estimaciones mejoran conforme avanza el proceso de entrenamiento y logra estabilizar sus errores alrededor de los 32 puntos, mientras que SARSA asciende hasta casi los 200 en los últimos episodios.

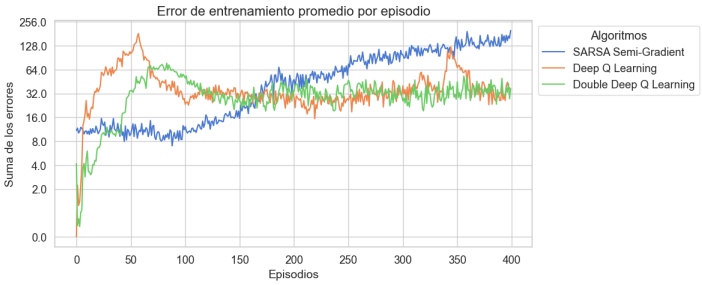


Figura 3.3: Promedio de errores de entrenamiento de los métodos aproximados

Análisis crítico

1. **SARSA Semi-Gradiente**: La restricción de la hipótesis lineal impide capturar las interacciones no lineales entre las variables de estado (posición, velocidad, ángulo y velocidad angular se correlacionan de forma compleja en CartPole). Adicionalmente, los 32 768 pesos reciben actualizaciones que solo afectan a $m = 4$ entradas en cada paso, mientras que DQN actualiza densamente sus más de 4000 parámetros vía *backpropagation*.

2. **DQN**: La no-linealidad de ReLU permite representar Q^* con precisión suficiente en 400 episodios. El Experience Replay estabiliza el gradiente estocástico al decorrelacionar las muestras. La principal limitación teórica es el sesgo de sobreestimación positivo acumulado en la propagación de Bellman.

3. **Double DQN**: El desacoplamiento Selección–Evaluación mediante la red target reduce el sesgo de sobreestimación. Sin embargo, la **inercia de la red target** ($\tau = 100$ pasos) ralentiza la convergencia inicial en el horizonte de 400 episodios. Con horizontes más largos ($T_{\text{ep}} > 1\,000$), la reducción del sesgo favorece estadísticamente a Double DQN, como documenta la literatura [9].

3.5. Conclusiones

Esta sección sintetiza los hallazgos principales del estudio experimental sobre métodos aproximados de Aprendizaje por Refuerzo, extrae las lecciones teóricas y prácticas derivadas de los resultados, discute las limitaciones del trabajo y propone líneas de investigación futuras.

3.5.1. Resumen de principales resultados

El experimento comparó tres familias de métodos aproximados sobre el entorno `CartPole-v1` con un protocolo de $N_{\text{runs}} = 10$ ejecuciones y $T_{\text{ep}} = 400$ episodios. Los resultados evidencian un ordenamiento consistente con las predicciones teóricas: DQN $\sim$ Double DQN $>$ SARSA Semi-Gradiente.

Se extraen cinco conclusiones clave:

1. **Los métodos aproximados generalizan en espacios continuos.** La sustitución de la tabla $\mathbf{Q}$ por un aproximador paramétrico $\hat{q}(\cdot;\boldsymbol{\theta})$ permite aprender en $\mathcal{S} \subseteq \mathbb{R}^4$, algo imposible con representación tabular.
2. **La capacidad del aproximador determina la calidad de la política.** La brecha entre SARSA y DQN refleja la diferencia en familia de hipótesis: la combinación lineal de tiles no puede representar las interacciones no lineales que sí captura ReLU.
3. **El Experience Replay es condición crítica de estabilidad.** El muestreo uniforme $\mathcal{B} \overset{\text{i.i.d.}}{\sim} \mathcal{D}$ convierte el problema en regresión supervisada, eliminando el sesgo de la correlación temporal entre transiciones consecutivas.
4. **La red target regulariza el problema no estacionario de los targets.** La copia periódica $\boldsymbol{\theta}^- \leftarrow \boldsymbol{\theta}$ cada $\tau = 100$ pasos congela la distribución de targets, convirtiendo la optimización no estacionaria en una cuasi-estacionaria localmente.
5. **El ordenamiento empírico depende del horizonte.** La ventaja teórica de Double DQN sobre DQN (sesgo de sobreestimación reducido) solo se manifiesta plenamente para $E > 1\,000$; en 400 episodios, la inercia de la red target ralentiza la convergencia inicial.

3.5.2. Limitaciones del estudio

A pesar de los resultados satisfactorios, el presente trabajo presenta las siguientes limitaciones que deben tenerse en cuenta al interpretar las conclusiones tal como muestra la tabla 3.5.

3.5.3. Líneas de trabajo futuro

1. Mayor horizonte de entrenamiento ($E > 1\,000$) para confirmar la superioridad de Double DQN sobre DQN.
2. Decaimiento de ε (*epsilon decay*): $\varepsilon(t) = \varepsilon_{\text{mín}} + (\varepsilon_0 - \varepsilon_{\text{mín}})e^{-\lambda t}$ para reducir exploración progresivamente.
3. Optimización bayesiana de hiperparámetros mediante *Optuna* o *Ax*.
4. Entornos de mayor complejidad como LunarLander-v2 ($\mathbb{R}^8$), BipedalWalker ($\mathbb{R}^{24}$) o MuJoCo.
5. Arquitecturas avanzadas: Dueling DQN, Prioritized Experience Replay, Rainbow.
6. Análisis estadístico formal: *bootstrapped confidence intervals* al 95 %, test de Wilcoxon con corrección de Bonferroni.
7. Transferencia de aprendizaje: uso de los pesos $\boldsymbol{\theta}$ preentrenados como inicialización en entornos relacionados.

Tabla 3.5: Limitaciones del estudio experimental

Limitación	**Impacto**
Horizonte reducido ($E = 400$)	Convergencia incompleta; ningún algoritmo alcanza 1000
Entorno único (`CartPole-v1`)	Rankings relativos no generalizables a otros entornos
Hiperparámetros fijos (sin búsqueda)	Resultados comparativos, no óptimos absolutos
Sin intervalos de confianza formales	No se cuantifica la varianza entre runs estadísticamente
Política ε constante ($\varepsilon = 0.1$)	No se explota la política aprendida en fases tardías

3.5.4. Reflexión final

Este capítulo demuestra que la transición de los métodos tabulares a los métodos aproximados no es meramente una extensión técnica, sino un cambio cualitativo en la naturaleza del problema de aprendizaje: de la memorización de una tabla finita a la **inferencia de una función** sobre un espacio continuo. Los tres algoritmos estudiados representan tres puntos del espectro de complejidad del aproximador —lineal, red *shallow*, red con target estabilizada— y sus diferencias de rendimiento ilustran de forma concreta el compromiso entre **capacidad expresiva**, **eficiencia muestral** y **estabilidad del entrenamiento**, algo que es central en la investigación moderna de Aprendizaje por Refuerzo Profundo.

Bibliografía

[1] Sutton, R.S. y Barto, A.G (2018). *Reinforcement Learning: An Introduction*. The MIT Press Cambridge, Second Edition.

[2] Auer, P., Cesa-Bianchi, N. y Fischer, P (2002). *Finite-time Analysis of the Multiarmed Bandit Problem*. Machine Learning 47, 235–256.

[3] Watkins, Christopher y Dayan, Peter. (1992). Technical Note: Q-Learning. Machine Learning. 8. 279-292. 10.1007/BF00992698.

[4] Rummery, G. y Niranjan, Mahesan. (1994). On-Line Q-Learning Using Connectionist Systems. Technical Report CUED/F-INFENG/TR 166.

[5] Van Hasselt, Hado. (2010). Double Q-learning. 2613-2621.

[6] Mnih, V. et al. (2015). Human-level control through deep reinforcement learning. *Nature*, 518, 529–533.

[7] Silver, D. et al. (2017). Mastering the game of Go without human knowledge. *Nature*, 550, 354–359.

[8] Bellman, R. (1957). *Dynamic Programming*. Princeton University Press.

[9] van Hasselt, H., Guez, A. & Silver, D. (2016). Deep Reinforcement Learning with Double Q-learning. *AAAI-16*.